

Comprendre la campagne d'expérimentation sur la fidélité des approximations de la matrice de Fisher

Un déroulé complet, sans raccourcis techniques

Septembre 2026

Résumé

Ce document explique, en partant de zéro, ce que mesure la campagne d'expérimentation décrite dans `docs/reports/plan_exp_draft.md` (et son brouillon original `plan_exp_draft_v0.md`). Le projet AdaFisher, et les quatre variantes construites autour de lui dans ce dépôt (K-FAC, EKFAC, TKFAC, TEKFAC), reposent tous sur une idée simple mais jamais vraiment vérifiée expérimentalement : au lieu de calculer exactement la « courbure » du problème d'apprentissage — une quantité bien trop grosse pour être manipulée telle quelle sur un réseau de neurones réaliste — on la remplace par une version très compressée, bon marché à calculer et à inverser. La question posée par cette campagne est directe : *à quel point cette version compressée ressemble-t-elle à l'originale, et où précisément perd-elle le plus d'information ?* Ce document reconstruit, sans supposer de connaissances préalables, tous les objets mathématiques en jeu, explique pourquoi chaque expérience est nécessaire, et rapporte ce qui a déjà été mesuré.

Table des matières

1	Introduction et mise en contexte	3
1.1	D'où vient ce projet	3
1.2	Ce que ce document est, et n'est pas	3
2	Les notions de base, construites depuis zéro	4
2.1	Pourquoi s'intéresser à autre chose que le gradient	4
2.2	Le problème d'échelle, et pourquoi on approxime	4
2.3	La matrice de Fisher : une courbure liée à la vraisemblance	4
2.4	Deux façons de faire la moyenne : vraie Fisher et Fisher empirique	5
2.5	Résumé intuitif	6
3	Le « zoo » des approximations : comment on compresse la courbure	6
3.1	Le bloc-diagonal exact : ignorer les couches entre elles	6
3.2	K-FAC : factoriser en produit de Kronecker	6
3.3	EKFAC : corriger K-FAC dans sa propre base	7
3.4	TKFAC et TEKFAC : une autre factorisation, qui préserve l'énergie totale	7
3.5	La diagonale : le compromis le plus radical	8
3.6	Le témoin diagonal libre : Adam	8
3.7	Un cas particulier important : les couches de normalisation	8
3.8	Les autres cas particuliers : <i>embeddings</i> , <i>LoRA</i> , la tête	9
4	Les trois sources du signal rétropropagé	9

5	Le design expérimental : croiser structure, source et référence	9
5.1	Le tableau à trois entrées	9
5.2	L'erreur totale se décompose-t-elle proprement ? (Q3)	10
5.3	La décomposition propre aux couches à poids partagés	11
6	Les trois régimes de calcul, dictés par la mémoire disponible	11
6.1	L'astuce commune : voir F comme un produit de deux petites choses	11
6.2	Régime A — tout former explicitement	11
6.3	Régime B — ne jamais former la grande matrice	12
6.4	Régime C — sans jamais former de matrice du tout	12
6.5	Décision prise pour cette campagne	12
7	Les modèles étudiés, et le rôle de chacun	13
8	Les deux protocoles de mesure : ce qui est possible contre ce qui se passe réellement	14
8.1	Protocole P1, structurel : la meilleure version possible	14
8.2	Protocole P2, opérationnel : ce que l'optimiseur utilise vraiment	14
8.3	Le problème du « réchauffage » (<i>re-warm</i>)	14
9	Amortissement et échelle : comment traiter le paramètre λ	15
10	Le plancher de bruit : à partir de quand une différence est-elle réelle ?	16
10.1	Pourquoi c'est nécessaire	16
10.2	Comment on le mesure	16
10.3	Sondes d'entraînement contre sondes de validation	16
11	Les points de contrôle, les « bras » de comparaison, et les graines aléatoires	16
11.1	Une trajectoire d'entraînement, cinq instantanés	16
11.2	Un piège déjà rencontré et corrigé	17
11.3	L'état des graines aléatoires	17
12	Les métriques, une par une	17
12.1	M1 — l'écart entrée par entrée (Frobenius)	17
12.2	M2 — la pire direction (norme spectrale)	18
12.3	M3 — la divergence de Kullback–Leibler entre deux gaussiennes (perte de Stein)	18
12.4	M4 — le conditionnement (vitesse de convergence prévisible)	18
12.5	M5 — l'efficacité du pas naturel (ρ), la métrique la plus directement pratique	18
12.6	M6 — le recouvrement des directions dominantes	19
12.7	M7 — le biais d'indépendance et la part du partage de poids	19
12.8	M8 — le couplage entre couches différentes	19
12.9	Comment calculer M3 et M5 sans jamais former de matrice $P \times P$ (régime B)	20
13	Les hypothèses et questions, remises en contexte	20
14	Ce qui a déjà été mesuré	22
14.1	Le premier calcul dense complet (modèle A1, sur la totalité des 55 000 images d'entraînement de MNIST)	23
14.2	Le blocage de l'auto-encodeur MNIST	23
14.3	Le zoo complet des approximations sur le modèle A1 (lot 2)	24
14.4	Les modèles à poids partagés : convolutions et attention (lot 3)	25
14.5	Un problème numérique corrigé en cours de route : les valeurs propres exactement nulles	27

15 Feuille de route : les lots d'implémentation restants	28
16 Risques identifiés et façons de les traiter	29
17 Ce que la campagne produira, et à quel coût	29
17.1 Le format des résultats	29
17.2 Les figures prévues	29
17.3 Le budget de calcul	30
18 Glossaire rapide	30

1 Introduction et mise en contexte

1.1 D'où vient ce projet

Le dépôt dans lequel s'inscrit ce plan développe un optimiseur, **AdaFisher**, et cinq façons différentes (« modes ») de construire l'ingrédient central dont il a besoin à chaque pas d'entraînement : une estimation de la courbure du problème, qui sert à corriger la direction et la taille du pas de descente de gradient. Les cinq modes (**diag**, **kfac**, **ekfac**, **tkfac**, **tekf**) ont déjà été implémentés, testés unitairement et comparés entre eux sur plusieurs bancs d'essai (un auto-encodeur MNIST, des CNN et des transformeurs sur CIFAR-10, etc.). Ce travail d'implémentation — huit « lots » numérotés dans `docs/reports/plan.md` — a répondu à une première question : *les cinq modes sont-ils correctement implémentés, au sens où chacun calcule bien ce que sa formule mathématique prescrit ?* La réponse est oui, vérifiée par des centaines de tests.

Mais une question plus profonde reste ouverte, et c'est elle que ce nouveau plan attaque : *ces formules elles-mêmes sont-elles de bonnes approximations de la quantité qu'elles prétendent approcher ?* Un mode peut être implémenté à la perfection tout en étant, dans l'absolu, une très mauvaise idée si l'hypothèse mathématique sur laquelle il repose ne tient pas sur les réseaux réels. C'est exactement le genre de question que l'article original d'AdaFisher n'a testée que très partiellement : sa propre validation (annexe B.2 du papier) se limite à un petit réseau jouet, une « vraie Fisher » obtenue par tirage aléatoire (Monte-Carlo) plutôt que calculée exactement, et une seule mesure (une erreur moyenne entre deux diagonales de matrices). La campagne décrite ici reprend ce travail depuis le début, en le rendant beaucoup plus rigoureux : calcul exact (et non approché) des quantités de référence, huit types de couches différents, plusieurs mesures complémentaires, et un « plancher de bruit » qui dit explicitement à partir de quand une différence mesurée cesse d'être significative.

1.2 Ce que ce document est, et n'est pas

Ce document **n'introduit aucune idée nouvelle** : il traduit, sans jargon et avec les justifications mathématiques complètes, le contenu de `plan_exp_draft.md` (la version actuellement suivie par le code, notée « v1 » ci-dessous) et de son brouillon original en français, `plan_exp_draft_v0.md` (noté « v0 »), qui définit les questions et hypothèses de recherche dans leur forme la plus explicite. Là où v1 a resserré le périmètre de v0 (moins de modèles, moins de métriques dans un premier temps, un sous-ensemble d'hypothèses priorisé), ce document le signale clairement plutôt que de mélanger les deux versions silencieusement.

Une distinction à garder en tête tout du long : cette campagne **ne modifie rien** au code de l'optimiseur (`src/adafisher_modes/`) ni aux bancs d'essai (`benchmarks/`). Elle construit un paquet complètement séparé, `fisher_ref/`, qui se contente de *lire* les modèles entraînés (leurs poids sauvegardés à différents instants de l'entraînement) et de calculer, sur ces instantanés

figés, des quantités de référence exactes auxquelles comparer chaque approximation. C’est un instrument de mesure, pas une nouvelle méthode d’entraînement.

2 Les notions de base, construites depuis zéro

2.1 Pourquoi s’intéresser à autre chose que le gradient

Entraîner un réseau de neurones consiste à ajuster ses paramètres θ pour rendre une fonction de perte $\ell(\theta)$ aussi petite que possible. La méthode la plus simple, la descente de gradient, ne regarde que la pente locale de cette fonction : à chaque pas, on avance dans la direction où la perte diminue le plus vite, avec un pas dont la taille (le taux d’apprentissage) est la même dans toutes les directions.

Le problème est que la perte n’est presque jamais également « raide » dans toutes les directions de l’espace des paramètres. Imaginons un paysage en forme de vallée très allongée et étroite : la direction perpendiculaire à la vallée est très raide (un petit déplacement fait beaucoup remonter la perte), la direction le long de la vallée est presque plate. Un pas de gradient bien réglé pour ne pas diverger dans la direction raide sera ridiculement petit dans la direction plate, et l’entraînement progresse alors très lentement. L’information qui manque au gradient seul, c’est la **courbure** : comment la pente elle-même change quand on se déplace. C’est exactement ce que mesure la matrice hessienne (ou une de ses cousines, voir plus bas) : une matrice carrée de taille $P \times P$ (P = nombre de paramètres), dont l’entrée (i, j) dit à quel point la pente selon le paramètre i change quand on bouge le paramètre j .

Connaître cette matrice permettrait de calculer, en un seul pas bien orienté et bien dimensionné, ce que le gradient seul mettrait des centaines de petits pas hésitants à approcher. C’est la promesse des méthodes dites de *second ordre* (par opposition au gradient, qui est de *premier ordre*), dont la méthode du gradient naturel — l’ancêtre conceptuel direct d’AdaFisher — fait partie.

2.2 Le problème d’échelle, et pourquoi on approxime

Le problème pratique est immédiat : pour un réseau de quelques millions de paramètres, la matrice de courbure complète a des millions au carré d’entrées — bien trop pour tenir en mémoire, et de toute façon impossible à inverser (l’opération dont on a besoin pour transformer le gradient en un « bon » pas) en un temps raisonnable. Toutes les méthodes pratiques de second ordre — y compris AdaFisher et ses quatre variantes — sont donc des **compromis** : elles ne gardent qu’une petite partie de l’information contenue dans cette énorme matrice, sous une forme bon marché à stocker et à inverser. La question de cette campagne est de mesurer, précisément, ce que chaque compromis perd.

2.3 La matrice de Fisher : une courbure liée à la vraisemblance

Il existe plusieurs façons raisonnables de définir « la courbure » d’un problème d’apprentissage. La famille étudiée ici tourne autour de la **matrice d’information de Fisher**, notée F , qui a un statut particulier pour les modèles entraînés par maximum de vraisemblance (ce qui couvre la quasi-totalité des cas pratiques : classification par entropie croisée, régression par erreur quadratique).

Concrètement, pour un réseau qui transforme une entrée x_n en une sortie z_n (les « logits » en classification), puis produit une distribution de probabilité sur les labels possibles, $r(y | z_n)$, la

matrice de Fisher s'écrit

$$F = \frac{1}{N} \sum_{n=1}^N J_n^\top \Lambda_n J_n, \quad J_n = \frac{\partial z_n}{\partial \theta}, \quad \Lambda_n = \text{Cov}_{y \sim r(\cdot | z_n)} [\nabla_z \log r(y | z_n)].$$

En mots : J_n décrit comment la sortie du réseau bouge quand on bouge chaque paramètre (c'est une information purement architecturale, calculable sans référence aux labels), et Λ_n décrit à quel point la distribution de probabilité que le modèle produit lui-même est sensible à un changement de sa propre sortie. Pour une classification softmax à C classes de probabilités $p_n = (p_{n,1}, \dots, p_{n,C})$, cette matrice Λ_n a une forme fermée très simple, $\Lambda_n = \text{diag}(p_n) - p_n p_n^\top$: c'est la matrice de covariance d'un vecteur one-hot tiré selon p_n .

Un fait mathématique important, que ce plan cite systématiquement, est que cette matrice de Fisher coïncide exactement avec une autre courbure très utilisée en pratique, la matrice de Gauss–Newton généralisée (*GGN*), **à condition** que la fonction de perte utilisée soit liée à r par ce qu'on appelle un « lien canonique » (Martens, 2020, section 9) : c'est vrai pour l'entropie croisée sur des logits, l'entropie croisée binaire sur des logits, et l'erreur quadratique — c'est-à-dire pour tous les cas que ce dépôt utilise réellement. C'est ce qui permet au plan de parler indifféremment de « Fisher vraie » et de « GGN » pour les modèles étudiés.

2.4 Deux façons de faire la moyenne : vraie Fisher et Fisher empirique

La formule ci-dessus fait apparaître une espérance implicite : Λ_n moyenne sur *toutes les valeurs possibles* de y que le modèle jugerait plausibles pour l'entrée x_n , pondérées par la probabilité que le modèle leur attribue lui-même. C'est ce qui rend le calcul de F coûteux : en pratique, on ne peut approcher exactement cette moyenne sur C classes qu'en faisant C passes de rétropropagation par exemple (une par classe possible), ou en utilisant une formule fermée quand elle existe (le cas softmax ci-dessus).

Il existe une alternative beaucoup moins chère : au lieu de moyenner sur toutes les étiquettes imaginables, on utilise **uniquement** l'étiquette réellement observée y_n pour chaque exemple. On obtient alors ce qu'on appelle la **Fisher empirique** :

$$\hat{E} = \frac{1}{N} \sum_n \nabla_\theta \ell_n \nabla_\theta \ell_n^\top, \quad \ell_n = -\log r(y_n | z_n).$$

Cette quantité ne coûte qu'**une seule** passe de rétropropagation par exemple (celle qu'on fait de toute façon pour calculer le gradient d'entraînement), contre C pour la vraie Fisher. C'est la raison structurelle pour laquelle *tous* les optimiseurs pratiques — AdaFisher inclus — n'ont jamais accès, dans la boucle d'entraînement réelle, qu'à une version de type « Fisher empirique », jamais à la vraie Fisher.

Le problème est que F et $\hat{E}$ ne sont **pas** la même quantité, et peuvent différer énormément, même quand le modèle est déjà bien entraîné (Kunstner, Balles & Hennig, 2019) : à l'optimum, le gradient d'un exemple individuel ne s'annule pas forcément, alors que la vraie distribution du modèle, elle, devient très proche de la vraie distribution des données. Mesurer précisément cet écart, indépendamment de toute approximation supplémentaire, est le premier axe de mesure de cette campagne (voir la question Q1, section 5).

2.5 Résumé intuitif

À retenir

- La courbure exacte d'un réseau de neurones est une matrice bien trop grosse pour être manipulée directement : toute méthode pratique en garde une version compressée.
- Pour les modèles entraînés par maximum de vraisemblance, la courbure naturelle est la matrice de Fisher F , qui coïncide (sous une condition presque toujours vérifiée ici) avec la matrice de Gauss–Newton.
- F (Λ moyennée sur les étiquettes imaginées par le modèle lui-même) est chère à calculer ; $\hat{E}$ (Fisher empirique, qui n'utilise que l'étiquette réelle) est bon marché mais biaisée. Les optimiseurs pratiques n'ont jamais accès qu'à une version de type $\hat{E}$.
- Cette campagne calcule, sur des instantanés figés de modèles réels, F et $\hat{E}$ de manière **exacte** (sans aucune structure imposée), pour servir de référence à laquelle comparer chaque approximation utilisée en pratique.

3 Le « zoo » des approximations : comment on compresse la courbure

Pour chaque couche ℓ du réseau (une couche linéaire, une convolution, une couche de normalisation, ...), on peut isoler le bloc de la matrice de courbure qui ne concerne que les paramètres de cette couche. Notons $a_{n,t}$ l'entrée de la couche pour l'exemple n à la position partagée t (un token dans une séquence, une position spatiale dans une image — pour une couche sans partage de poids, il n'y a qu'une seule position), avec un 1 ajouté pour représenter le biais, et $g_{n,c,t}$ le gradient qui redescend jusqu'à la sortie de cette couche (avant sa non-linéarité), pour la colonne c associée à une des sources décrites plus bas. Le gradient de la couche pour cet exemple et cette colonne est $\mathcal{G}_{n,c} = \sum_t g_{n,c,t} a_{n,t}^\top$, et le bloc exact de courbure de la couche s'écrit

$$B_\ell = \frac{1}{N} \sum_{n,c} \text{vec}(\mathcal{G}_{n,c}) \text{vec}(\mathcal{G}_{n,c})^\top.$$

Ce bloc B_ℓ est *exact* : c'est le sous-bloc diagonal de F (si c parcourt les colonnes de la source « vraie Fisher », voir section 4) ou de $\hat{E}$ (source « empirique ») qui concerne la couche ℓ . Toutes les approximations ci-dessous en sont des simplifications.

3.1 Le bloc-diagonal exact : ignorer les couches entre elles

La première simplification, la plus douce, consiste à garder B_ℓ **exactement** pour chaque couche, mais à supposer que deux couches différentes n'ont aucune corrélation de courbure entre elles (les blocs hors diagonale, entre couches, sont mis à zéro). C'est un compromis raisonnable si l'information qui circule entre couches est réellement faible — une hypothèse que cette campagne teste directement (métrique M8 et hypothèse HF5, plus bas).

3.2 K-FAC : factoriser en produit de Kronecker

L'idée centrale de K-FAC (Martens & Grosse, 2015) est de remarquer que, *si* les entrées $a_{n,t}$ de la couche et les gradients $g_{n,c,t}$ qui en ressortent étaient statistiquement indépendants, le bloc B_ℓ se factoriserait exactement en un **produit de Kronecker** $A \otimes G$ de deux matrices bien plus petites : une matrice A qui ne décrit que les corrélations entre les entrées de la couche, et une

matrice G qui ne décrit que les corrélations entre les gradients de sortie. Concrètement,

$$A = \frac{1}{NT} \sum_{n,t} a_{n,t} a_{n,t}^\top, \quad G = \frac{1}{N} \sum_{n,c,t} g_{n,c,t} g_{n,c,t}^\top, \quad K_{\text{KFAC}} = A \otimes G.$$

L'intérêt est majeur : si la couche a d_{in} entrées et d_{out} sorties, le bloc exact a $(d_{\text{in}} d_{\text{out}})^2$ entrées, alors que A et G n'en ont que $d_{\text{in}}^2 + d_{\text{out}}^2$ — et surtout, l'inverse d'un produit de Kronecker se calcule comme le produit de Kronecker des inverses, $(A \otimes G)^{-1} = A^{-1} \otimes G^{-1}$, ce qui rend le préconditionnement abordable à chaque pas d'entraînement.

Le prix de cette économie est double, et les deux effets sont mesurés séparément par cette campagne :

- pour une couche à **poids partagés** (une convolution, ou une couche linéaire appliquée à chaque position d'une séquence comme dans l'attention), il faut d'abord décider comment traiter les différentes positions t : soit les traiter comme des exemples indépendants qu'on empile (*expand*, exact pour un réseau linéaire profond selon Eschenhagen et al., 2023), soit résumer d'abord chaque exemple par une somme sur ses positions avant de corrélérer (*reduce*, exact quand l'agrégation finale est une moyenne pondérée, par exemple un *pooling* moyen). Le choix *expand/reduce* n'est pas neutre, et son erreur est mesurée par la décomposition « partage de poids » de la section 5 ;
- même en ignorant le partage de poids, l'hypothèse d'**indépendance** entre entrées et gradients de sortie n'est vraie qu'en général de façon approximative. L'écart entre le bloc exact B_ℓ et sa version « produit de Kronecker le plus proche » est l'**erreur d'indépendance**, mesurée directement (métrique M7).

3.3 EK-FAC : corriger K-FAC dans sa propre base

K-FAC choisit implicitement une direction privilégiée pour chaque paramètre de la couche : les vecteurs propres du produit de Kronecker $A \otimes G$, c'est-à-dire les produits croisés des vecteurs propres de A (notés Q_A) et de G (notés Q_B). Le raffinement EK-FAC (George et al., 2018) garde exactement cette même base de directions, mais réajuste la « force » attribuée à chacune de ces directions de façon **optimale**, en la recalculant directement à partir des données :

$$s_{ij} = \frac{1}{N} \sum_{n,c} \left[(Q_G^\top g_{n,c} Q_A)_{ij} \right]^2.$$

Cette correction est peu coûteuse (elle ne change pas la base, seulement des coefficients), et elle garantit par construction que la trace totale du bloc approché reste égale à celle du bloc exact B_ℓ — une propriété que K-FAC lui-même n'a pas. Intuitivement, EK-FAC dit : « je garde les mêmes directions que K-FAC, parce que les calculer autrement serait cher, mais je ne suppose plus que leur importance suit exactement le produit des deux facteurs séparés — je la mesure directement ».

3.4 TKFAC et TEKFAC : une autre factorisation, qui préserve l'énergie totale

TKFAC (Gao et al., 2021) part d'une factorisation différente, construite pour garantir exactement une propriété précise : la trace du bloc de courbure approché (une mesure de son « énergie » totale, la somme de ses valeurs propres) est **égale**, par construction, à la trace du bloc exact B_ℓ — et ce sans avoir besoin de recalculer quoi que ce soit après coup, contrairement à EK-FAC. TEKFAC applique à TKFAC le même raffinement qu'EKFAC applique à K-FAC : garder la base propre issue de TKFAC, mais réajuster optimalement l'importance de chaque direction.

3.5 La diagonale : le compromis le plus radical

La version la plus économique de toutes consiste à ne garder que la diagonale du bloc de courbure, c'est-à-dire à supposer qu'il n'existe **aucune** corrélation entre deux paramètres différents, même à l'intérieur d'une même couche — chaque paramètre est traité indépendamment de tous les autres. C'est ce que fait le mode `diag` d'AdaFisher, avec une propriété remarquable : la diagonale d'un produit de Kronecker $A \otimes G$ est exactement le produit de Kronecker des diagonales, $\text{diag}(A) \otimes \text{diag}(G)$. Autrement dit, l'estimateur « brut » d'AdaFisher **est** la diagonale de K-FAC. Il existe donc deux chemins différents pour l'obtenir : le calculer directement comme diagonale du bloc exact B_ℓ , ou le calculer comme diagonale du produit de Kronecker $A \otimes G$. Ces deux chemins ne coïncident pas exactement, et leur écart, entrée par entrée, vaut précisément

$$\mathbb{E}[a_j^2 g_i^2] - \mathbb{E}[a_j^2] \mathbb{E}[g_i^2] = \text{Cov}(a_j^2, g_i^2),$$

c'est-à-dire le **bias d'indépendance restreint à la diagonale seule** — une quantité que la campagne mesure directement, car elle est distincte de la perte, bien plus large, de toute l'information hors diagonale.

Ce qu'AdaFisher utilise réellement dans l'optimiseur n'est pas cette diagonale « brute », mais une version encore transformée pour la rendre numériquement stable et utilisable pas après pas : normalisée entre 0 et 1 (« min-max »), lissée dans le temps par une moyenne mobile exponentielle (EMA) qui combine l'estimation du pas courant avec la mémoire des pas précédents, et enfin amortie par un petit terme λ pour éviter de diviser par une valeur trop proche de zéro. Cette version pleinement opérationnelle est appelée « AF-op » dans le plan, par opposition à la diagonale « brute », « AF-raw ». La différence entre les deux est précisément l'objet du deuxième protocole de mesure, P2 (section 8).

3.6 Le témoin diagonal libre : Adam

Un point de comparaison utile, gratuit, est le second moment que l'optimiseur Adam maintient déjà pour chaque paramètre (la moyenne mobile des carrés du gradient). Ce n'est pas conçu comme une approximation de courbure, mais des travaux récents (Li, Dangel, Tam & Raffel, 2025) soutiennent que toute méthode diagonale « sophistiquée » devrait au moins faire mieux que ce témoin gratuit, sans quoi la sophistication ne sert à rien. Les campagnes de bancs d'essai de ce dépôt font tourner Adam et AdamW comme deux des sept « bras » comparés systématiquement (voir section 11), ce qui fournit ce témoin sans coût supplémentaire.

3.7 Un cas particulier important : les couches de normalisation

Les couches de normalisation (`LayerNorm`, `GroupNorm`, `BatchNorm2d` en mode évaluation) ont deux paramètres par canal, une échelle γ et un décalage β , appliqués à une entrée déjà normalisée $\hat{x}_{n,t}$ (de moyenne nulle et de variance unité). Leurs gradients exacts par exemple sont simples : $\nabla_\gamma = \sum_t g_t \odot \hat{x}_t$ et $\nabla_\beta = \sum_t g_t$.

Un résultat mathématique établi pour AdaFisher (la proposition 3.1 de son article) montre que le bloc de courbure exact d'une telle couche **n'a pas** la structure « produit de Kronecker » des couches précédentes, mais une structure différente, dite de Hadamard (un produit terme à terme entre deux matrices, plutôt qu'un produit de Kronecker). Faire entrer cette structure différente dans le même moule que les quatre approximations précédentes (qui attendent toutes un produit de Kronecker $A \otimes G$) demande un choix de conception explicite — un petit facteur scalaire choisi pour être la meilleure approximation possible (au sens des moindres carrés, « optimale au sens de Frobenius ») de la vraie structure de Hadamard. Ce choix, documenté et délibéré, est un des points que la campagne vérifie chiffre en main (c'était l'hypothèse HF4 de v0, voir plus bas), et le repère a en réalité déjà été largement tranché analytiquement au moment

d'implémenter les modes (lot 5) : ce que cette campagne ajoute est la mesure *empirique*, sur un bloc exact $2C \times 2C$ toujours calculable (même dans le régime le plus contraint en mémoire), de l'écart entre trois variantes : le bloc exact complet (avec le couplage entre γ et β), le bloc exact « séparé » (ce que fait AdaFisher, qui jette ce couplage), et la structure de Hadamard approchée réellement implémentée.

3.8 Les autres cas particuliers : *embeddings*, *LoRA*, la tête

- **Position/embedding** (`pos_embed`, `cls_token`) : ce sont des paramètres « nus », qui n'appartiennent à aucune couche standard reconnue par les hooks de l'optimiseur — leur courbure y est donc traitée comme l'identité (aucune information de courbure n'est utilisée pour eux), un choix qu'il faut savoir expliciter plutôt que de le découvrir accidentellement.
- **LoRA** (adaptateurs de *fine-tuning* léger, non encore couverts par les modèles actuellement construits) : un adaptateur s'écrit $W_0 + BA$, un produit de deux petites matrices. La courbure présente naturellement un couplage entre les deux adaptateurs A et B (le produit BA est bilinéaire), que la métrique M8 (couplage inter-blocs) est destinée à quantifier si ce cas est ajouté à la campagne.
- **La tête de classification** : c'est une couche linéaire ordinaire, sans partage de poids, mais avec une particularité : la matrice $\Lambda(p)$ qui pondère la sortie dépend elle-même de l'entrée a de cette couche (par l'intermédiaire de $p = \text{softmax}(z)$). L'hypothèse d'indépendance entre entrée et gradient, sur laquelle repose K-FAC, y est donc fausse « par construction » — c'est un point de mesure informatif plutôt qu'un problème à corriger.

4 Les trois sources du signal rétropropagé

Dans la formule de B_ℓ ci-dessus, l'indice c parcourt un ensemble de « colonnes » qui dépend de la source σ choisie pour le vecteur g — c'est-à-dire de la façon dont on choisit de faire la moyenne sur les étiquettes possibles évoquée en section 2.4 :

- **type-2** (la vraie Fisher) : $c = 1, \dots, C$, une colonne par classe possible, en utilisant la racine fermée du softmax $S_n[:, c] = \sqrt{p_c}(e_c - p_n)$, qui vérifie exactement $S_n S_n^\top = \Lambda_n$. C'est coûteux (C rétropropagations par exemple) mais exact, sans aucun tirage aléatoire.
- **Monte-Carlo** (MC_K) : au lieu de considérer toutes les classes, on en tire K au hasard selon la distribution p_n du modèle lui-même, et on utilise le vecteur $K^{-1/2}(p_n - e_{\tilde{y}})$ pour chaque tirage $\tilde{y}$. C'est une estimation **stochastique** de la vraie Fisher (elle y converge quand K augmente, à la vitesse $O(K^{-1/2})$ classique de tout estimateur Monte-Carlo), moins chère quand $K \ll C$. Le facteur $K^{-1/2}$ est indispensable : sans lui, l'estimateur ne converge tout simplement pas vers la bonne quantité.
- **empirique** : $c = 1$, une seule colonne, en utilisant directement l'étiquette réellement observée, $p_n - e_{y_n}$. C'est ce que tout optimiseur pratique utilise réellement.

Chaque structure d'approximation (K-FAC, EK-FAC, ...) peut être appliquée à chacune de ces trois sources : « **structure** » et « **source** » sont deux axes de variation indépendants, et c'est précisément ce croisement qui constitue le cœur du design expérimental (section suivante).

5 Le design expérimental : croiser structure, source et référence

5.1 Le tableau à trois entrées

Pour chaque couche et chaque instant de l'entraînement, on calcule un écart $d(S(\sigma), R)$ entre :

- une **structure** S : bloc complet, bloc-diagonal exact, K-FAC-expand, K-FAC-reduce, EK-FAC, TK-FAC, diagonale-Kronecker d’AdaFisher, diagonale exacte, (et pour les normalisations : Hadamard ou la version réellement implémentée) ;
- une **source** σ : type-2, Monte-Carlo à $K \in \{1, 4, 16\}$ tirages, ou empirique ;
- une **référence** R : la vraie Fisher F ou la Fisher empirique exacte $\hat{E}$.

Certaines cellules de ce tableau à trois entrées répondent directement, chacune, à une question précise :

Cellule mesurée	Ce qu’elle isole	Question
$d(\hat{E}, F)$	l’erreur de source seule (aucune structure imposée)	Q1
$d(S(\text{type-2}), F)$	l’erreur de structure seule, contre la vraie Fisher	Q2
$d(S(\text{emp}), \hat{E})$	l’erreur de structure seule, contre la Fisher empirique	Q2
$d(S(\text{emp}), F)$	le chemin réellement emprunté par AdaFisher en pratique	Q3
$d(S(\text{MC}_K), F)$	structure + bruit d’échantillonnage Monte-Carlo	HF6
tout ce qui précède, par type de couche	tout, désagrégé	Q4

En clair : **Q1** demande « si je ne fais *aucune* approximation de structure, combien est-ce que je perds simplement en remplaçant les vraies étiquettes imaginées par le modèle par les étiquettes réelles observées ? ». **Q2** demande l’inverse : « à source fixée, combien est-ce que je perds en compressant la structure (diagonale, Kronecker, ...) ? ». **Q3** combine les deux dans l’ordre où AdaFisher les subit réellement en pratique (structure compressée *et* source empirique), et compare le résultat à la vraie Fisher. **Q4** demande si les réponses aux trois questions précédentes dépendent du type de couche (une couche linéaire simple n’est pas forcément dans le même cas qu’une tête d’attention ou qu’une normalisation).

5.2 L’erreur totale se décompose-t-elle proprement ? (Q3)

Une question naturelle, une fois Q1 et Q2 mesurées séparément, est de savoir si l’erreur totale du chemin réel (Q3) est simplement leur somme. Pour une vraie distance mathématique (comme l’erreur de Frobenius ou l’erreur spectrale, voir section suivante), l’inégalité triangulaire garantit *seulement* une borne supérieure :

$$\underbrace{d(S(\text{emp}), F)}_{\text{erreur totale}} \leq \underbrace{d(\hat{E}, F)}_{\text{erreur de source}} + \underbrace{d(S(\text{emp}), \hat{E})}_{\text{erreur de structure}} .$$

Ce que cette campagne rapporte, c’est l’**écart entre les deux membres** de cette inégalité : si l’erreur totale mesurée est très inférieure à la somme des deux erreurs séparées, cela signifie que les deux sources d’erreur ont tendance à *se compenser* l’une l’autre (une bonne nouvelle accidentelle) ; si elle est proche de la somme, les deux erreurs *s’additionnent* sans se gêner ni s’annuler. Pour la métrique de divergence de Kullback–Leibler (section 12, métrique M3), qui n’est pas une distance mathématique au sens strict (elle n’est pas symétrique), on se contente de rapporter les trois termes côte à côte, sans chercher à les additionner.

5.3 La décomposition propre aux couches à poids partagés

Pour une couche à poids partagés (une convolution appliquée à chaque position spatiale, une couche linéaire appliquée à chaque position d’une séquence dans l’attention), le passage du bloc exact B_ℓ à K-FAC-expand se fait en deux étapes distinctes, dont chacune a un coût mesurable séparément :

$$B_\ell \xrightarrow[\text{croisés } t \neq t' \text{ ignorés}]{\text{erreur de **partage de poids**}} B_\ell^{\text{expand}} \xrightarrow{\text{erreur d'**indépendance**}} \text{K-FAC-expand.}$$

La première flèche consiste à ignorer les corrélations entre deux positions différentes de la même couche (par exemple, entre deux pixels différents d’une même image traversant la même convolution) ; la seconde est l’hypothèse d’indépendance entre entrées et gradients de sortie déjà décrite pour K-FAC en général. Les mesurer séparément permet de savoir laquelle des deux hypothèses coûte le plus cher sur une architecture donnée — une information directement exploitable pour décider où investir un effort de modélisation plus fin. C’est maintenant fait, et la réponse est nette : sur les trois modèles à poids partagés mesurés, la première flèche emporte à elle seule une fraction médiane de 0,971 du bloc exact, la seconde 0,45 à 0,60 de ce qui reste (section 14.4).

6 Les trois régimes de calcul, dictés par la mémoire disponible

Calculer F ou $\hat{E}$ de façon exacte, sans aucune structure imposée, revient à former une matrice $P \times P$ pleine — le même problème d’échelle que celui qui a motivé, en premier lieu, toutes les approximations décrites ci-dessus. La campagne le résout en distinguant trois régimes de calcul selon la taille du réseau étudié.

6.1 L’astuce commune : voir F comme un produit de deux petites choses

Le point de départ commun est une écriture différente de F . On empile, pour chaque exemple sonde n et chaque colonne c de la racine $\Lambda_n^{1/2}$ (par exemple les C colonnes de la racine softmax fermée S_n décrite plus haut), une ligne $N^{-1/2}(\Lambda_n^{1/2} e_c)^\top J_n$ dans une grande matrice $U \in \mathbb{R}^{m \times P}$, avec $m = N \cdot (\text{rang de la racine})$. On a alors exactement

$$F = U^\top U.$$

Chaque ligne de U ne coûte qu’une seule rétropropagation d’un vecteur de sortie. C’est cette écriture qui permet de choisir, selon P et m , la stratégie de calcul la moins coûteuse.

6.2 Régime A — tout former explicitement

Quand le réseau est assez petit (P de l’ordre de quelques dizaines de milliers de paramètres, la limite exacte étant $P_{\max} \approx \sqrt{B/24}$ où B est la mémoire GPU disponible en octets — soit environ 28 900 paramètres sur 20 Go, ou seulement 20 400 sur les 10 Go du plus petit découpage matériel utilisé pour les entraînements), on peut se permettre de calculer $U^\top U$ pour de vrai et d’obtenir une matrice $P \times P$ que l’on stocke intégralement en mémoire, en double précision (*fp64*) — une précision plus élevée que celle utilisée pendant l’entraînement, parce que le spectre des valeurs propres de F s’étend sur plus de huit ordres de grandeur, et la précision simple habituelle (*fp32*) ne suffit pas à représenter fidèlement les valeurs propres les plus petites. C’est le seul régime où l’on peut calculer **tout** exactement : le spectre complet, les blocs entre couches différentes, toutes les métriques sans aucune approximation supplémentaire. Le prix est le nombre limité de modèles qui rentrent dans cette catégorie, et le fait qu’il faut aussi choisir un nombre d’exemples sondes N suffisamment grand pour que F puisse être de rang plein (sinon toute métrique qui

passer par un inverse serait dominée par le petit terme d’amortissement ajouté pour rendre cet inverse possible, plutôt que par le contenu réel de F).

6.3 Régime B — ne jamais former la grande matrice

Pour des modèles plus grands (des centaines de milliers de paramètres), former U dans son intégralité serait déjà trop coûteux en mémoire. Le régime B ne forme **jamais** U ni F entiers : il garde, couche par couche, un « Gram » beaucoup plus petit, $K_\ell = U_\ell U_\ell^\top \in \mathbb{R}^{m \times m}$ (une matrice de taille liée au nombre d’exemples sondes plutôt qu’au nombre de paramètres de la couche), et les statistiques brutes de couche (les $a_{n,t}$ et $g_{n,c,t}$ eux-mêmes). Une identité algébrique classique (l’identité de Woodbury, la même qui rend les filtres de Kalman efficaces) permet alors de calculer, sans jamais former de matrice $P \times P$, la plupart des métriques d’intérêt : normes de Frobenius, traces, inverses amortis, divergences de Kullback–Leibler. Deux façons pratiques de construire ce Gram par couche existent, à choisir couche par couche selon celle qui coûte le moins cher : la matérialiser directement, ou passer par une astuce dite « *ghost* » qui l’obtient sans jamais former de gradient individuel par exemple, empruntée aux méthodes de confidentialité différentielle.

Le prix de ce régime est différent de celui du régime A : F , construite à partir d’un nombre fini m d’exemples sondes, a un **rang** au plus m , qui peut être bien plus petit que P . Les métriques qui ne passent pas par un inverse (Frobenius, direction) restent exactes ; celles qui en passent par un mélange de deux effets qu’il faut soigneusement démêler : l’erreur réelle dans la partie de l’espace que F « voit » réellement, et une courbure **inventée** par une approximation de rang plein comme K-FAC là où F n’a strictement aucune information (son noyau). C’est pourquoi toute conclusion tirée du régime B est systématiquement présentée en fonction d’un paramètre d’amortissement λ balayé sur plusieurs ordres de grandeur, et confrontée, quand c’est possible, au régime A qui n’a pas ce problème.

6.4 Régime C — sans jamais former de matrice du tout

Pour les très grands modèles (un ResNet-50 complet, par exemple), même les Grams par couche du régime B deviendraient trop gros. Le régime C ne calcule **que des produits** de la matrice de courbure par un vecteur (Fv), obtenus en une passe avant plus une passe arrière, sans jamais matérialiser aucune matrice intermédiaire. Cela donne accès à des quantités comme la norme spectrale ou la trace, calculées par des algorithmes itératifs (Lanczos, gradient conjugué), au prix d’une petite erreur stochastique ou d’approximation contrôlée. Ce régime, bien que prévu dans le plan, est optionnel et repoussé aux derniers lots d’implémentation : il apporte un modèle de plus au prix d’un jeu de métriques plus restreint et de moins bonne précision.

6.5 Décision prise pour cette campagne

Un outil externe existant, **curvlinops**, aurait pu fournir directement les opérateurs « sans matrice » du régime C et servir d’oracle indépendant pour valider le reste du code. La décision, prise et documentée au premier lot d’implémentation, est de **ne pas l’adopter** (il n’est disponible ni dans l’environnement du projet ni sur le cluster de calcul utilisé) : à la place, un oracle indépendant plus simple, construit directement à partir des outils de différentiation automatique de PyTorch, a été écrit spécifiquement pour vérifier le code de la campagne, et il concorde avec l’implémentation principale à $3,8 \times 10^{-16}$ près sur des vecteurs aléatoires — une précision largement suffisante pour servir de garde-fou.

7 Les modèles étudiés, et le rôle de chacun

Le tableau suivant reprend les huit dossiers de bancs d’essai (déjà construits et testés pour les huit lots d’implémentation de l’optimiseur) réellement utilisés par cette campagne, avec le nombre exact de paramètres de chacun et le régime de calcul dans lequel il tombe.

Id	Modèle	P	Régime	Types de couches	Rôle dans la campagne
A1	MLP + Layer-Norm (MNIST)	26 634	A	Linear sans partage, Layer-Norm, tête	le cas témoin, le plus propre pour la théorie de K-FAC
A2	CNN + Group-Norm/BatchNorm (CIFAR-10)	24 458	A	Convolution (partage spatial), normalisation, tête	reproduit le banc jouet de l’article AdaFisher, mais contre une Fisher exacte
A3	ViT miniature (CIFAR-10)	21 098	A	<i>patch-embed</i> , attention QKV fusionnée, MLP, LayerNorm, tête	un cadre « <i>reduce</i> » propre (agrégation par moyenne)
B0	Auto-encodeur MNIST	2 837 314	B	8 couches linéaires, sigmoïdes	le banc principal du projet, celui où les cinq modes se bloquent (section 14.2)
B1	CCT-2/3×2 (CIFAR-10)	283 723	B	tokenizer convolutionnel, transformeur, LayerNorm, <i>pooling</i> de séquence	le modèle propre à l’article AdaFisher
B2	ResNet-20 (CIFAR-10)	269 722	B	convolution, BatchNorm	le petit ResNet de la famille testée par AdaFisher
B4	ViT à largeur réaliste (CIFAR-10)	2 693 578	B, en flux	transformeur $d = 192$, 6 blocs	vérifie que les conclusions tirées sur A3/B1 tiennent à une largeur réaliste
C	ResNet-50 (CIFAR-10)	23 520 842	C unique-ment	convolutions profondes, BatchNorm	optionnel, régime sans matrice seulement

Deux modèles envisagés dans le brouillon initial (un modèle de langage miniature de type GPT, et un modèle de *fine-tuning* léger par LoRA sur RoBERTa) ne sont **délibérément pas construits** pour l’instant : ce sont les deux seuls cas qui porteraient des types de couches encore absents du reste de la campagne (un *embedding* de tokens, une attention causale, des adaptateurs LoRA), et ils restent les premiers candidats pour un futur dossier de modèle, une fois que la carte principale sera stabilisée sur les huit modèles ci-dessus.

Trois précisions structurelles à garder en tête, héritées directement de la façon dont ces modèles ont été construits pour les besoins de l’optimiseur, avant même cette campagne :

- Dans A2, la variante **GroupNorm** n’est **pas** un type de couche reconnu par l’optimiseur AdaFisher : ses paramètres reçoivent, dans l’optimiseur, un traitement équivalent à « pas de courbure du tout » (préconditionneur identité). Ce n’est pas un oubli mais un choix délibéré du banc, qui donne à la campagne un cas de contrôle gratuit : une normalisation dont le bloc exact est parfaitement calculable, mais dont la contrepartie « opérationnelle » côté optimiseur est simplement absente.
- Dans A3, B1 et B4, l’attention expose une seule couche linéaire fusionnée qui calcule en un bloc les trois projections *query*, *key* et *value* — un choix d’implémentation (hérité de la bibliothèque utilisée) qui empêche, pour l’instant, de mesurer séparément l’erreur de K-FAC sur chacune des trois projections individuellement ; seule la version « bloc fusionné, entrée partagée » est mesurable avec les modèles actuels.
- Les vecteurs de position (**pos_embed**) et le jeton de classification (**cls_token**), là où ils existent, sont des paramètres « nus » n’appartenant à aucune couche reconnue : ils sont bien mis à jour pendant l’entraînement, mais leur courbure y est traitée comme l’identité

— un fait que le registre de classification des paramètres de la campagne doit explicitement reconnaître plutôt que de le laisser passer inaperçu.

8 Les deux protocoles de mesure : ce qui est possible contre ce qui se passe réellement

Une même approximation (disons K-FAC) peut être évaluée de deux façons très différentes, et cette campagne les distingue systématiquement.

8.1 Protocole P1, structurel : la meilleure version possible

Dans le protocole P1, on prend un jeu de poids θ figé (un instantané de l’entraînement), un ensemble fixe d’exemples sondes, et on calcule chaque approximation **sans** aucune des approximations supplémentaires que l’optimiseur ajoute en pratique pour tenir sur la durée d’un entraînement complet : pas de lissage temporel (pas de moyenne mobile), pas de renormalisation min-max, et un amortissement λ balayé systématiquement plutôt que fixé à une seule valeur. P1 répond à la question : « *dans le meilleur des cas, quelle est la fidélité maximale que cette famille d’approximation peut atteindre ?* »

8.2 Protocole P2, opérationnel : ce que l’optimiseur utilise vraiment

Dans le protocole P2, on va lire, directement dans l’état interne de l’optimiseur réel en train de s’exécuter, la version de la courbure telle qu’elle sert **réellement** à corriger chaque pas : après le lissage temporel (moyenne mobile exponentielle, avec un poids de 0,92 donné à l’historique et 0,08 au facteur du mini-batch le plus récent, à chaque mise à jour tous les 100 pas), après la renormalisation min-max (pour le mode **diag**), avec l’amortissement réellement utilisé en entraînement ($\lambda = 10^{-3}$), avec les couches de normalisation en mode entraînement (donc avec un comportement légèrement différent du mode évaluation utilisé pour toutes les références exactes), et avec le *dropout* actif. On compare ensuite cet état réel aux références exactes, calculées au **même** θ . P2 répond à la question : « *que se passe-t-il réellement, une fois tous les compromis pratiques empilés les uns sur les autres ?* »

La comparaison entre P1 et P2 est directement ce qui permet de répondre à l’hypothèse HF7 (plus bas) : si P2 est nettement pire que P1, la faiblesse ne vient pas de la structure mathématique choisie (Kronecker, diagonale, ...) mais des compromis d’ingénierie ajoutés autour d’elle (le lissage temporel, la normalisation, l’amortissement) — une conclusion qui oriente la recherche vers des pistes très différentes (par exemple, un lissage temporel plus fidèle à la géométrie du problème, plutôt qu’une nouvelle structure d’approximation).

8.3 Le problème du « réchauffage » (*re-warm*)

Un obstacle pratique complique P2 : les instantanés sauvegardés pendant l’entraînement ne contiennent que les **poids** du réseau, pas la mémoire interne de l’optimiseur (les moyennes mobiles accumulées). Pour mesurer P2 à un instant donné de l’entraînement, il faut donc repartir des poids sauvegardés, reconstruire un optimiseur neuf, et le laisser tourner un certain nombre de pas supplémentaires pour que sa mémoire interne se « réchauffe » et redevienne représentative de ce qu’elle aurait été si l’entraînement avait continué sans interruption — avant de faire la mesure.

Combien de pas faut-il pour ce réchauffage ? La règle de mise à jour de la moyenne mobile multiplie l’ancien état par 0,92 et y ajoute 0,08 fois la nouvelle observation, à chaque intervalle de 100 pas (TCov). Après k intervalles, la contribution de *tout ce qui existait avant le réchauffage* est

multipliée par $0,92^k$ — ou, lu autrement, la composition des poids successifs donne $0,92, 0,92 \times 0,08, \dots$, c'est-à-dire que **92 % de l'état opérationnel n'est jamais que le facteur du tout dernier mini-batch**. Un point de correction important, découvert en creusant cette arithmétique : la longueur de réchauffage nécessaire n'est pas fixée par la condition naïve « $0,08^k$ doit être négligeable devant 1 », mais par la condition, plus stricte, « $0,08^k$ doit être négligeable devant λ » — parce que chaque optimiseur initialise sa mémoire avec la matrice identité au tout premier pas, et qu'un réchauffage trop court laisse subsister une trace de cette identité initiale qui agit comme un **amortissement parasite**, s'ajoutant au λ voulu. À $k = 3$ intervalles (300 pas), cette trace parasite ($0,08^3 \approx 5 \cdot 10^{-4}$) vaut déjà la moitié de $\lambda = 10^{-3}$, faussant le préconditionneur appliqué de 12 à 87 % selon le mode. Il faut au moins $k = 10$ intervalles (1000 pas) pour que cette contamination retombe à un niveau négligeable.

Résultat mesuré — fidélité du réchauffage, `cnm_gn_cifar`, 11 septembre 2026

Une expérience dédiée a comparé, à partir d'un même point θ^* issu d'un vrai entraînement de 2 000 pas, trois optimiseurs neufs réchauffés sur des ordres de mini-batches différents : deux avec les poids figés (leur écart mutuel donne le **plancher de bruit** propre à l'estimateur, dû uniquement au tirage aléatoire des mini-batches), un avec les poids qui continuent d'évoluer (donnant le terme de « désuétude », c'est-à-dire l'effet du déplacement de θ pendant le réchauffage). Trois conclusions, mesurées et non supposées :

1. Après un réchauffage suffisant (au moins 10 intervalles, soit 1000 pas), chaque mode se retrouve à seulement 1,3 à 4,5 fois son propre plancher de bruit — c'est-à-dire pratiquement indiscernable d'une deuxième réalisation aléatoire de la même expérience. C'est ce qui rend inutile de relancer toute la campagne d'entraînement uniquement pour conserver l'état de l'optimiseur : le réchauffage suffit.
2. Le terme de désuétude (l'effet du déplacement des poids pendant le réchauffage) est, lui, invisible : il est du même ordre que le bruit lui-même, ce qui n'a rien de surprenant puisque la moyenne mobile ne donne de toute façon jamais plus de 8 % de poids à l'historique lointain.
3. Les facteurs bruts eux-mêmes restent, quelle que soit la longueur de réchauffage, à 2 à 4 fois leur propre plancher de bruit : ce ne sont, par nature, que des estimateurs à forte variance, et relancer un entraînement complet ne ferait que tirer une nouvelle réalisation de cette même variance, pas une valeur « plus vraie ».

Un interrupteur optionnel, désactivé par défaut, permet désormais de sauvegarder directement la mémoire de l'optimiseur en plus des poids dans les futurs instantanés (`--checkpoint-optimizer-state`), pour éviter ce réchauffage dans les prochaines campagnes. Il n'a pas été activé rétroactivement, car cela obligerait à recommencer les entraînements déjà réalisés — ce qui, au vu du point (1) ci-dessus, n'apporterait pas d'information nouvelle par rapport à un simple réchauffage.

9 Amortissement et échelle : comment traiter le paramètre λ

Toute métrique qui passe par un inverse (impossible à calculer directement sur une matrice qui a des valeurs propres nulles ou proches de zéro) a besoin d'un petit terme d'amortissement λ ajouté sur la diagonale avant l'inversion. Plutôt que de fixer une seule valeur de λ (ce qui reviendrait à figer arbitrairement un compromis), la campagne balaie systématiquement $\lambda = \alpha \bar{\lambda}$, avec $\bar{\lambda} = \text{tr}(R)/P$ (une échelle naturelle liée à la taille moyenne des valeurs propres de la référence) et $\alpha \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$, et présente les résultats sous forme de courbes plutôt que de points isolés. La valeur $\lambda = 10^{-3}$ réellement utilisée par AdaFisher en entraînement n'est ajoutée qu'en protocole P2, car elle n'a de sens que pour des facteurs déjà renormalisés entre 0 et 1 par le min-max — un traitement propre au mode `diag` qui n'a pas d'équivalent dans les

quatre autres modes.

Pour les approximations qui ne prétendent, par construction, respecter aucune échelle absolue (la diagonale brute d'AdaFisher, ou le second moment libre d'Adam), on rapporte en plus une version « recalée au mieux », en multipliant l'approximation par le facteur scalaire $c^* = \langle R, K \rangle_F / \|K\|_F^2$ qui minimise l'écart de Frobenius — de façon à ne pas pénaliser injustement une méthode pour un simple facteur d'échelle qu'elle n'a jamais prétendu respecter.

10 Le plancher de bruit : à partir de quand une différence est-elle réelle ?

10.1 Pourquoi c'est nécessaire

F et $\hat{E}$ sont elles-mêmes calculées sur un nombre fini N d'exemples sondes : ce sont donc des estimations, avec leur propre variabilité statistique. Si l'on compare deux approximations et que l'écart mesuré entre elles est plus petit que la variabilité naturelle de F elle-même (c'est-à-dire l'écart qu'on obtiendrait simplement en changeant l'échantillon d'exemples utilisés pour la calculer), cette différence ne veut rien dire : elle pourrait tout aussi bien disparaître, voire s'inverser, avec un autre tirage d'exemples.

10.2 Comment on le mesure

On coupe l'ensemble des exemples sondes en deux moitiés aléatoires, on calcule F séparément sur chaque moitié, et on mesure l'écart entre les deux résultats — en répétant l'opération sur 20 partitions aléatoires différentes pour obtenir un intervalle statistique à 95 %. **Toute différence entre deux approximations qui n'excède pas ce plancher n'est simplement pas interprétable.** C'est un principe de lecture des résultats, pas une hypothèse en soi (dans le brouillon initial, cette règle porte le nom d'hypothèse HF8 ; dans le plan actuellement suivi, elle est reclassée comme une *règle de lecture* systématique plutôt que comme une hypothèse à trancher une fois pour toutes).

10.3 Sondes d'entraînement contre sondes de validation

Un deuxième axe de comparaison consiste à utiliser deux ensembles d'exemples sondes différents : des exemples que le modèle a effectivement vus pendant son entraînement, et des exemples qu'il n'a jamais vus (un ensemble de validation, tiré de la même partition aléatoire que celle réellement utilisée par l'entraînement, pour garantir que « entraînement » désigne bien les points sur lesquels le modèle a réellement appris). L'écart entre F et $\hat{E}$ pourrait très bien se comporter différemment sur ces deux ensembles — c'est exactement ce que teste l'hypothèse HF1, détaillée section 13.

11 Les points de contrôle, les « bras » de comparaison, et les graines aléatoires

11.1 Une trajectoire d'entraînement, cinq instantanés

Pour chaque modèle, les bancs d'essai de ce dépôt sauvegardent déjà, à des fractions fixes de l'entraînement complet (0 %, 1 %, 10 %, 50 % et 100 %), un instantané des poids du réseau. Ces fractions sont calculées relativement à une trajectoire de référence **commune**, partagée par toutes les stratégies d'entraînement comparées (ce que le plan appelle des « bras » : sept au total — les cinq modes de courbure, plus Adam, plus AdamW), de sorte qu'un instantané «

à 50 % » représente bien la même *quantité d'entraînement* quel que soit le bras, même si les différentes stratégies n'avancent pas forcément à la même vitesse en temps réel. C'est cette trajectoire commune, avec ses cinq instantanés, qui constitue l'axe principal le long duquel la campagne mesure comment la fidélité des approximations *évolue* au fil de l'entraînement (plutôt qu'une mesure ponctuelle à un seul instant, comme le fait l'article original d'AdaFisher).

11.2 Un piège déjà rencontré et corrigé

Une subtilité pratique, découverte lors d'une première campagne d'entraînement sur le cluster de calcul : pour un bras qui ne dispose que d'un budget de temps fixe plutôt que d'un nombre d'époques fixe, la fraction « réellement atteinte » à la fin de son propre entraînement peut être inférieure à 100 %. Un instantané marqué comme « non programmé » signale que ce n'est pas la fraction nominale de 100 % qui a été atteinte, mais simplement la fin propre à ce bras — une distinction essentielle pour ne pas comparer par erreur des quantités d'entraînement différentes sous une même étiquette.

11.3 L'état des graines aléatoires

Chaque expérience est en principe répétée sur plusieurs graines aléatoires différentes (des initialisations et des ordres de mini-batches différents), pour s'assurer qu'une conclusion ne tient pas à un seul tirage de chance. Au moment de la rédaction, la graine 0 est disponible pour les sept bras sur six des huit modèles ; des graines supplémentaires sont en cours d'exécution sur le cluster de calcul pour un sous-ensemble de bras (uniquement `diag` et `adamw`, les deux stratégies jugées les plus informatives pour cet axe). La règle de lecture explicite est que **tout résultat agrégé sur plusieurs graines doit annoncer le nombre de graines réellement disponibles au moment de la mesure**, plutôt que de supposer par avance un nombre cible qui n'est pas encore garanti.

12 Les métriques, une par une

Toutes les métriques ci-dessous comparent une référence exacte R (soit F , soit $\hat{E}$) à une approximation K , avec $X_\lambda = X + \lambda I$ pour désigner une matrice amortie. Elles sont choisies pour capturer des aspects **complémentaires** de la ressemblance entre deux matrices : aucune métrique seule ne suffirait, comme l'illustre l'exemple suivant. Deux matrices peuvent avoir un écart de Frobenius très faible parce qu'elles se ressemblent presque partout, tout en étant radicalement différentes dans une seule direction très importante — c'est exactement ce que les métriques ci-dessous (M1 à M8) sont chacune conçues pour détecter ou, au contraire, pour ignorer.

12.1 M1 — l'écart entrée par entrée (Frobenius)

$$e_F = \frac{\|R - K\|_F}{\|R\|_F}, \quad \cos_F = \frac{\langle R, K \rangle_F}{\|R\|_F \|K\|_F}, \quad e_F^* = \sqrt{1 - \cos_F^2}.$$

C'est la mesure la plus simple : on traite les deux matrices comme deux tableaux de nombres et on mesure leur écart quadratique moyen, normalisé par la taille de la référence — l'équivalent de comparer deux images pixel par pixel. Le produit scalaire de Frobenius $\langle R, K \rangle_F$ donne un « cosinus » entre les deux matrices, qui ignore leur échelle relative (e_F^* , la version « à échelle optimale », retire ce facteur d'échelle avant de mesurer l'écart résiduel). Cette métrique a un défaut connu : elle est **dominée** par les plus grandes valeurs propres, et peut donc masquer une erreur sévère concentrée dans une direction de faible amplitude mais importante pour l'optimisation.

12.2 M2 — la pire direction (norme spectrale)

$$e_2 = \frac{\|R - c^*K\|_2}{\|R\|_2}.$$

Alors que M1 moyenne l'erreur sur toutes les directions, M2 ne regarde que la **direction où l'erreur est la plus grande** (la norme spectrale, ou norme d'opérateur, d'une matrice symétrique est sa plus grande valeur propre en valeur absolue). C'est une mesure de robustesse dans le pire des cas, complémentaire de M1.

12.3 M3 — la divergence de Kullback–Leibler entre deux gaussiennes (perte de Stein)

$$D_\lambda(K\|R) = \frac{1}{2} \left[\text{tr}(K_\lambda R_\lambda^{-1}) - P - \log \det(K_\lambda R_\lambda^{-1}) \right].$$

Cette métrique demande un peu plus de mise en contexte. Toute matrice de courbure amortie R_λ peut être vue comme la matrice de covariance *inverse* d'une distribution gaussienne hypothétique sur les déplacements de paramètres — c'est exactement l'objet mathématique que le gradient naturel utilise implicitement pour définir « la bonne direction, à la bonne échelle ». $D_\lambda(K\|R)$ est alors, tout simplement, la divergence de Kullback–Leibler classique entre les deux distributions gaussiennes associées à K_λ et R_λ — une façon standard, en théorie de l'information, de mesurer à quel point deux distributions de probabilité sont différentes.

L'intérêt décisif de cette métrique par rapport à M1/M2 est qu'elle est **invariante par reparamétrage linéaire** : si l'on change arbitrairement d'échelle ou de base de coordonnées pour les paramètres (un changement qui ne devrait rien changer au *comportement* d'un préconditionneur, seulement à sa représentation numérique), D_λ reste identique, alors que M1 et M2 changeraient. C'est donc le critère le plus fidèle à ce qui compte vraiment pour un préconditionneur : non pas « à quel point les nombres se ressemblent », mais « à quel point les deux définissent la même notion de distance dans l'espace des paramètres ». Un test de cohérence trivial mais rassurant : quand $K = R$, cette divergence vaut exactement zéro.

12.4 M4 — le conditionnement (vitesse de convergence prévisible)

$$\kappa_\lambda = \frac{\lambda_{\max}}{\lambda_{\min}} \text{ de } K_\lambda^{-1/2} R_\lambda K_\lambda^{-1/2}.$$

Cette métrique répond à une question très pratique : si l'on utilisait K comme préconditionneur dans une méthode itérative de type gradient conjugué pour résoudre un problème dont la vraie courbure est R , combien d'itérations faudrait-il ? La théorie classique de l'optimisation dit que ce nombre d'itérations croît comme $\sqrt{\kappa}$ (la racine carrée du rapport entre la plus grande et la plus petite valeur propre de l'opérateur « relatif » ci-dessus) : un κ proche de 1 signifie que K est un préconditionneur presque parfait pour R .

12.5 M5 — l'efficacité du pas naturel (ρ), la métrique la plus directement pratique

$$\rho(K) = \frac{(g^\top d)^2}{(d^\top R_\lambda d)(g^\top R_\lambda^{-1} g)}, \quad d = K_\lambda^{-1} g.$$

C'est probablement la métrique la plus intuitive et la plus directement liée à l'utilisation réelle d'une approximation de courbure en entraînement. Étant donné un gradient g , on calcule le pas

$d = K_\lambda^{-1}g$ que l'approximation *proposerait* de suivre. $\rho(K)$ compare la baisse de perte que ce pas d permettrait effectivement d'obtenir (dans un modèle quadratique local dont la vraie courbure est R) à la baisse de perte **optimale** qu'on obtiendrait avec la meilleure direction possible, $R_\lambda^{-1}g$. Par une inégalité de Cauchy–Schwarz, ρ est toujours compris entre 0 et 1 : $\rho = 1$ signifie que l'approximation, même compressée, donne exactement le même pas que la référence exacte ; ρ proche de 0 signifie que le pas proposé est presque inutile, voire contre-productif.

C'est cette métrique que la section 14.2 identifie comme la mesure la plus directement décisive pour comprendre l'échec observé du banc principal (l'auto-encodeur MNIST) : elle permet de demander directement, à l'endroit précis où l'entraînement se bloque, « le pas que l'optimiseur propose réellement est-il, ou non, encore une bonne approximation du pas idéal ? ».

12.6 M6 — le recouvrement des directions dominantes

$$\frac{\|V_R^\top V_K\|_F^2}{k}, \quad k \in \{C, 10C\}.$$

Les spectres de courbure des réseaux de neurones ont une propriété bien documentée dans la littérature : un petit nombre de directions (de l'ordre du nombre de classes C) concentrent l'essentiel de l'énergie, le reste étant beaucoup plus plat. M6 mesure si les quelques directions les plus importantes de l'approximation (V_K , ses vecteurs propres dominants) coïncident bien avec les quelques directions les plus importantes de la vraie référence (V_R) — indépendamment de ce qui se passe dans le reste, beaucoup plus plat, du spectre.

12.7 M7 — le biais d'indépendance et la part du partage de poids

Cette famille de mesures quantifie directement, plutôt que de le supposer, à quel point l'hypothèse d'indépendance sur laquelle repose K-FAC est vraie sur les réseaux réels : le rapport entre la deuxième et la première valeur singulière (proche de 0 si le bloc exact ressemble vraiment à un simple produit de Kronecker, plus grand sinon) et la part d'énergie totale qui n'est pas capturée par cette meilleure approximation en produit de Kronecker possible, obtenue par décomposition en valeurs singulières (SVD) d'un réarrangement particulier de la matrice B_ℓ^{expand} . Le même outil sert à isoler, séparément, la part perdue à cause du partage de poids ($\|B_\ell - B_\ell^{\text{expand}}\|_F / \|B_\ell\|_F$) et la part perdue en ne gardant que la diagonale du produit de Kronecker.

12.8 M8 — le couplage entre couches différentes

$$c_{\ell\ell'} = \frac{\langle K_\ell, K_{\ell'} \rangle_F}{\|K_\ell\|_F \|K_{\ell'}\|_F}.$$

Cette métrique quantifie la validité de l'approximation la plus radicale de toutes citées plus haut : le bloc-diagonal exact, qui suppose qu'aucune information de courbure ne circule entre deux couches différentes. C'est mathématiquement l'équivalent d'une mesure de similarité (une forme non centrée de ce qu'on appelle en apprentissage automatique une « CKA », *centered kernel alignment*) entre les « noyaux tangents » de deux couches — un $c_{\ell\ell'}$ proche de zéro confirmerait que les couches sont bien quasiment indépendantes en termes de courbure ; une valeur substantielle indiquerait, au contraire, une information réelle perdue par l'approximation bloc-diagonale.

12.9 Comment calculer M3 et M5 sans jamais former de matrice $P \times P$ (régime B)

Un point technique important, qui explique pourquoi le régime B (streamé, couche par couche) peut malgré tout calculer les métriques M3 (KL) et M5 (ρ), pourtant définies à partir d'inverses de matrices $P \times P$: en réécrivant $R = U^\top U$ avec $U \in \mathbb{R}^{m \times P}$ ($m \ll P$ possible), l'identité de Woodbury permet de remplacer chaque inverse $P \times P$ par un inverse $m \times m$, bien moins coûteux :

$$R_\lambda^{-1} = \lambda^{-1}(I - U^\top M^{-1}U), \quad M = \lambda I_m + UU^\top,$$

et de même pour le log-déterminant, via l'identité de Sylvester, $\log \det R_\lambda = P \log \lambda + \log \det M - m \log \lambda$. C'est le même principe mathématique qui rend les filtres de Kalman efficaces, appliqué ici à une matrice de courbure qui n'est, par construction, jamais que le produit d'un nombre fini d'exemples sondes.

13 Les hypothèses et questions, remises en contexte

Le brouillon original du plan (v0) formulait huit hypothèses **falsifiables** — chacune accompagnée explicitement de la condition qui la réfuterait, une exigence de rigueur scientifique délibérée. Le plan actuellement suivi par le code (v1) resserre l'ordre de priorité de ces huit hypothèses (certaines sont traitées en premier, d'autres reportées, une devient une simple règle de lecture plutôt qu'une question ouverte) ; les deux niveaux sont présentés ici ensemble, avec leur statut actuel.

HF1 — l'écart source-vs-vraie-Fisher se dégrade-t-il différemment sur l'entraînement et sur la validation ?

Énoncé : sur des exemples tirés de l'ensemble d'entraînement, l'écart entre la Fisher empirique et la vraie Fisher (mesuré par l'efficacité ρ) se dégrade au fil de l'entraînement ; sur des exemples de validation jamais vus, beaucoup moins. *Intuition* : sur des points déjà appris, le gradient d'un exemple individuel tend vers zéro d'une façon particulière qui n'a pas de raison de se produire sur des points non vus — c'est une forme du même phénomène de surapprentissage qui affecte la perte elle-même, mais appliqué ici à la courbure plutôt qu'à la valeur de la perte. *Réfutée si* les deux courbes (entraînement, validation) coïncident à l'intérieur du plancher de bruit. *Statut actuel* : priorisée dans v1 ; une première mesure sur le modèle A1 avait laissé un résultat ambigu entre les trois combinaisons possibles (entraînement/validation, entraînement/test, validation/test), faute d'un dénominateur commun enregistré pour les trois. Ce défaut d'instrumentation est corrigé — chaque écart est maintenant rapporté avec les deux normes qui le divisent — et la cause de l'ambiguïté est identifiée : la courbure mesurée sur des exemples jamais vus est réellement *plus grande* (de 30 à 60 % sur A1), parce qu'à mi-parcours le modèle est plus confiant sur ce qu'il a appris. L'hypothèse elle-même reste ouverte : aucun des calculs menés jusqu'ici n'a utilisé de sondes de validation, tous ayant porté sur des sondes d'entraînement.

HF2 — l'erreur de structure de K-FAC dépend-elle du type de couche, et *expand* contre *reduce* ?

Énoncé : l'erreur de structure de K-FAC est plus grande sur les couches à poids partagés (attention, convolution) que sur les couches simples, et le choix *reduce* (qui résume d'abord chaque exemple par une moyenne sur ses positions avant de corrélérer) est préférable en classification d'image, l'inverse en modélisation du langage. *Réfutée si* l'écart entre les deux choix est sous le plancher de bruit. *Statut* : **tranchée** — **confirmée sur les trois modèles à poids partagés** (section 14.4). Les deux volets le sont : l'erreur de structure est bien plus grande sur les

couches à poids partagés que sur la tête de classification du même modèle, et *reduce* l’emporte sur *expand* dans 14/15, 15/15 et 40/45 des cas, très au-dessus du plancher de bruit. Deux réserves accompagnent ce verdict, toutes deux mesurées : l’erreur ainsi mesurée provient *d’abord* du partage de poids lui-même (97 % de l’écart) et non de l’hypothèse d’indépendance ; et le classement s’inverse si l’on juge non plus la fidélité mais la qualité du pas produit (ρ), ce qui arrive dans environ 70 % des cas. Le volet « l’inverse en modélisation du langage » reste, lui, non testé : aucun modèle de langage n’est encore construit dans ce dépôt.

HF3 — l’hypothèse d’indépendance d’AdaFisher a-t-elle un coût mesurable sur la seule diagonale ?

Énoncé : l’écart entre les deux chemins vers la diagonale d’AdaFisher (décrit section 4.6, une covariance $\text{Cov}(a_j^2, g_i^2)$) dépasse 10 % sur les couches de normalisation et sur l’attention. *Réfutée si* les deux chemins coïncident à 5 % près. *Statut* : **tranchée — confirmée sur les quatre modèles déjà traités** (sections 14.3 et 14.4). Le raccourci coûte 0,86–0,90 sur les convolutions et 0,83–0,97 sur les couches d’attention et de MLP du transformeur en fin d’entraînement, contre 0,25–0,41 sur les têtes de classification : un ordre de grandeur au-delà du seuil de 10 % que l’hypothèse s’était fixé. La seule exception connue est la première couche du modèle A1, et elle s’explique entièrement par les pixels toujours nuls de MNIST.

HF4 — le couplage γ - β ignoré par AdaFisher dans les normalisations est-il négligeable ?

Énoncé : dans une couche de normalisation, le terme croisé entre les deux paramètres γ et β — que la proposition 3.1 de l’article AdaFisher exclut explicitement — porte une fraction non négligeable de l’énergie du bloc exact complet. *Statut* : déjà en grande partie résolue **analytiquement**, au moment même de construire les cinq modes de l’optimiseur (avant cette campagne) : le bloc exact d’une normalisation a une structure de Hadamard, pas de Kronecker, et le petit terme croisé y a été délibérément conservé plutôt que forcé à zéro. Cette mesure est maintenant faite sur les quatre modèles : le terme croisé porte environ deux tiers de l’énergie du bloc exact sur le modèle A1, 0,65–0,70 sur le convolutif et 0,39–0,62 sur le transformeur — non négligeable partout, donc.

Un résultat imprévu du même lot (section 14.4) : la forme de Hadamard décrit correctement la *direction* de la courbure d’une normalisation, mais en sous-estime la *taille* d’un facteur 20 à 38 dès qu’un poids est appliqué à plusieurs positions (contre un facteur 0,95, c’est-à-dire aucun écart, sur le modèle sans partage). Un estimateur corrigé pour les normalisations doit donc corriger l’échelle, et pas seulement réintroduire le terme croisé. Et la formule effectivement livrée aujourd’hui dans le code se classe dernière des cinq lectures comparées, y compris une fois l’échelle neutralisée.

HF5 — l’information qui circule entre deux couches différentes compte-t-elle vraiment ?

Énoncé : ignorer entièrement les corrélations entre couches (l’approximation bloc-diagonale) ne coûte presque rien, aussi bien sur un CNN que sur un transformeur — sauf peut-être sur le couplage spécifique entre les projections *query* et *key* de l’attention. *Réfutée si* l’écart est grand sur le transformeur. *Statut* : première réponse partielle, et elle va dans le sens de l’hypothèse — mais dans une direction inattendue. Le couplage mesuré entre deux couches (section 14.4) est *plus faible* là où les poids sont partagés : au minimum 0,28 sur le transformeur et 0,44 sur le convolutif, contre 0,63 sur le perceptron A1. Ignorer les corrélations inter-couches est donc mieux justifié sur les architectures réalistes que sur le modèle jouet où la question avait d’abord

été posée. Le verdict complet demande encore la mesure directe de ρ pour l’approximation bloc-diagonale, et la séparation *query/key* que les modèles actuels ne permettent pas (leur projection est fusionnée).

HF6 — un tirage Monte-Carlo unique dégrade-t-il l’estimation de courbure ?

Énoncé : avec un seul tirage aléatoire ($K = 1$) et un petit *batch*, l’estimation de type Monte-Carlo est moins bonne (en ρ) que l’estimation purement empirique — un résultat déjà suggéré par des travaux antérieurs sur la Fisher empirique inversée. *Statut* : repoussée dans l’ordre de priorité de v1 (elle n’est testée qu’à travers les sources Monte-Carlo optionnelles, $K \in \{1, 4, 16\}$, qui ne sont pas la priorité des premiers lots).

HF7 — le classement « en pratique » (P2) est-il le même que le classement « dans l’idéal » (P1) ?

Énoncé : le classement des cinq approximations, mesuré dans les conditions réelles d’entraînement (P2), n’est **pas** le même que celui mesuré dans les conditions idéalisées (P1) — autrement dit, ce sont le lissage temporel et la renormalisation, pas la structure mathématique choisie, qui dominent l’erreur réellement subie par AdaFisher en pratique. *Statut* : priorisée dans v1 ; les mesures préliminaires de fidélité du réchauffage (section 7.4) vont déjà dans ce sens en creux — un réchauffage insuffisant peut, à lui seul, fausser le préconditionneur appliqué de plusieurs dizaines de pourcents, indépendamment de toute question de structure.

HF8 — une partie des différences mesurées est-elle simplement du bruit ?

Énoncé : à petite taille d’échantillon, une partie des différences apparentes entre approximations est en réalité indiscernable de la variabilité statistique propre à F elle-même. *Statut dans v1* : cette hypothèse n’est plus traitée comme une question à trancher une fois, mais promue en **règle de lecture systématique** appliquée à *tous* les résultats de la campagne (section 13 plus haut, le « plancher de bruit ») — un changement de statut logique plutôt qu’un abandon. *Constat pratique* : ce plancher dépend énormément du modèle et du nombre d’exemples sondes. Il valait 0,343 sur le modèle A1 (55 000 exemples), au point de rendre inexploitable un premier essai à 512 exemples ; il tombe à 0,005–0,039 sur les trois modèles du lot 3 (45 000 exemples), où les écarts mesurés lui sont dix à cent fois supérieurs. La règle reste indispensable — c’est elle qui avait invalidé l’essai à 512 exemples — mais elle ne mord pas sur les résultats du lot 3.

Les questions Q1 à Q5

Les questions Q1 à Q4 (déjà détaillées section 5) structurent le design factoriel lui-même — elles ne sont pas des hypothèses au sens strict (elles ne prédisent rien à l’avance), mais des axes de décomposition de l’erreur totale. La question **Q5** du brouillon original (« que devient le classement en conditions opérationnelles réelles ? ») est exactement ce que le protocole P2 (section 8) répond directement dans le plan actuel ; elle n’est donc plus listée séparément, ayant simplement été absorbée dans la définition même de P2.

14 Ce qui a déjà été mesuré

Cette section rassemble, en un seul endroit, tout ce que la campagne a déjà produit comme résultats concrets — par opposition au reste de ce document, qui décrit surtout ce qui est *prévu*.

14.1 Le premier calcul dense complet (modèle A1, sur la totalité des 55 000 images d'entraînement de MNIST)

Un premier calcul complet et exact de F et $\hat{E}$ (sans aucune structure imposée) a été mené sur le modèle A1, en utilisant **tout** l'ensemble d'entraînement de MNIST comme exemples sondes (55 000 images), sur une durée d'un peu plus d'une heure de calcul GPU. Plusieurs constats mesurés en sont sortis :

Résultats mesurés — le calcul dense A1

- Le plancher de bruit propre à F , estimé en coupant les 55 000 exemples en deux moitiés, vaut 0,2986 (soit un écart-type de bruit $\sigma_N = 0,1493$). L'écart mesuré entre la Fisher vraie et la Fisher empirique (Q1) vaut 0,3751, soit **2,51 fois** ce plancher de bruit : l'écart source-Fisher est donc réellement interprétable sur ce modèle, et non un artefact d'échantillonnage.
- Ce plancher de bruit décroît plus lentement que ce que prédit la théorie de l'estimation statistique classique : en passant de 4 000 à 55 000 exemples (un facteur 13,75), le bruit n'a diminué que d'un facteur 2,92, alors qu'un estimateur « idéal » aurait dû s'améliorer d'un facteur 3,71 ($\propto N^{-1/2}$). Autrement dit, MNIST n'offre plus beaucoup de marge d'amélioration supplémentaire par ce seul levier.
- Le rang réel de F (21 829 sur 26 634 paramètres possibles) et celui de $\hat{E}$ (18 342) montrent que, une fois le nombre d'exemples suffisamment grand, c'est la Fisher *empirique* qui est structurellement la plus déficiente en rang — un résultat plus net que ce que la seule théorie générale (Kunstner et al., 2019) permettait d'anticiper.
- Chaque déficience de rang a une explication structurelle précise : le bloc de la tête de classification a un noyau de dimension exactement $d_{\text{in}} + 1$ (ajouter une même constante à chaque logit ne change rien à la sortie du softmax — un fait mathématique, pas un artefact numérique) ; et le premier bloc du réseau a un noyau de dimension 4 579, presque exactement 32×143 , qui correspond aux pixels de bordure de MNIST, identiquement nuls sur toutes les images.
- Ce même premier bloc du réseau concentre à lui seul **63 %** de l'énergie totale (la trace) de F : sur ce modèle particulier, la courbure du problème d'apprentissage est massivement dominée par sa toute première couche, un fait qui doit conditionner l'interprétation de toute conclusion « par type de couche » tirée de ce modèle.

Ce même calcul a également mis au jour, en amont, un piège de conception facile à manquer : l'accroche technique utilisée par défaut pour capturer les gradients (`register_full_backward_hook`) ne se déclenche **pas** pour la toute première couche du réseau, dès lors que l'entrée du réseau elle-même ne demande pas de gradient — ce qui aurait silencieusement mis à zéro 94 % des paramètres du modèle A1 dans le calcul de référence, sans qu'aucune erreur ne soit levée. Le code de la campagne utilise donc une méthode différente et plus sûre pour capturer ces gradients, avec un test de non-régression dédié qui vérifie explicitement ce cas.

14.2 Le blocage de l'auto-encodeur MNIST

Le banc d'essai **principal** du projet (l'auto-encodeur MNIST, huit couches linéaires avec des sigmoïdes) présente une anomalie déjà identifiée avant même le début de cette campagne, et que la campagne est justement conçue pour aider à élucider :

Constat mesuré — le blocage de l’auto-encodeur

Les cinq modes de courbure (**diag**, **kfac**, **ekfac**, **tkfac**, **tekf**) se bloquent tous à une erreur quadratique moyenne d’environ 0,7085 dès les deux premières époques d’entraînement, et n’en bougent quasiment plus ensuite : entre 10 % et 100 % de la durée totale de l’entraînement, les paramètres ne se déplacent plus que de 0,2 à 0,5 %. Dans les mêmes conditions, Adam continue de progresser normalement jusqu’à une erreur de 0,4836, en déplaçant ses paramètres de 147 % sur la même période. Un *sweep* du terme d’amortissement λ sur quatre ordres de grandeur a déjà exclu ce paramètre comme cause principale : il ne modifie le résultat final que de moins de 1 % pour les quatre modes de type Kronecker.

Les deux causes restantes, non encore testées avant cette campagne, sont le taux d’apprentissage partagé par tous les modes, et **la géométrie même du produit de Kronecker $A \otimes B$ sur ce réseau particulier** — c’est-à-dire la possibilité que l’approximation en produit de Kronecker soit, sur ce réseau précis, structurellement mal adaptée (plutôt que simplement mal réglée). C’est précisément cette deuxième cause que la métrique M5 (ρ , l’efficacité du pas naturel) et le rapport $\|\tilde{F}^{-1}\hat{m}\|/\|\hat{m}\|$ (à quel point le pas préconditionné diffère en norme du gradient brut) sont conçus pour trancher directement, en comparant, au même point de blocage, la trajectoire produite par un mode de type Kronecker à celle produite par Adam. Les instantanés nécessaires à cette mesure existent déjà (ils ont été sauvegardés lors des campagnes d’entraînement précédentes) ; c’est la mesure la plus rapide et la plus décisive que le premier travail en régime B de cette campagne doit produire, précisément parce qu’elle répond à une question que le développement de l’optimiseur, à lui seul, ne peut actuellement pas trancher.

14.3 Le zoo complet des approximations sur le modèle A1 (lot 2)

Le deuxième lot a appliqué au modèle A1 l’ensemble des approximations décrites en section 3 et les métriques M1, M3, M5, M7, M8, à ses cinq instantanés d’entraînement et pour les deux sources de signal. Trois choses en sont ressorties.

D’abord, **le raccourci diagonal d’AdaFisher n’est pas gratuit** (HF3) : sur la toute première couche du réseau il ne coûte presque rien — mais pour une raison très particulière, à savoir que 130 des 784 pixels de MNIST sont identiquement nuls, et qu’une quantité toujours nulle ne peut corrélérer avec rien ; sur toutes les couches situées en aval, l’écart est réel et *croît* au fil de l’entraînement.

Ensuite, un résultat inattendu, qui n’était pas la question posée : **toutes les approximations se dégradent nettement au fil de l’entraînement** quand on les juge sur la qualité du pas qu’elles produiraient (métrique ρ). En début d’entraînement, les approximations structurées récupèrent 61 à 97 % du pas idéal ; à la fin, seulement 13 à 46 %, et les approximations diagonales tombent à une fraction de pourcent sur les premières couches. Une approximation presque exacte au départ est un mauvais substitut de la vérité à l’arrivée.

Enfin, une **correction apportée après coup** à la lecture de ce lot, et qu’il faut connaître avant de lire la suite. Sur les couches de normalisation, quatre lectures simplifiées avaient été comparées, et le classement obtenu avait été présenté comme un résultat. Le lot 3 a établi que trois des quatre comparaisons de ce classement sont en réalité des **théorèmes** — vraies d’avance, quelles que soient les données : la diagonale exacte est, par construction, la meilleure approximation diagonale possible au sens de Frobenius, et le bloc « séparé » est la meilleure matrice sans termes croisés. Seule la comparaison entre la forme de Hadamard et la diagonale exacte était une véritable mesure. De plus, la lecture présentée comme « la formule réellement utilisée par l’optimiseur » n’était pas cette formule, mais la diagonale de la forme de Hadamard. Les deux points sont corrigés dans le lot 3, qui calcule la formule réellement livrée en appelant

directement le code de l'optimiseur, et qui vérifie désormais les trois inégalités-théorèmes à l'exécution : leur violation serait un bogue, jamais un résultat.

14.4 Les modèles à poids partagés : convolutions et attention (lot 3)

Le troisième lot étend tout ce qui précède aux deux modèles où un même poids est appliqué à *plusieurs endroits* : le petit réseau convolutif A2 (chaque filtre balaie l'image, soit 1 024, 256 puis 64 positions selon la couche), dans ses deux variantes de normalisation, et le petit transformeur A3 (la même couche traite les 64 jetons de l'image). C'est la situation pour laquelle K-FAC a été généralisé, et celle que le modèle A1 — où chaque poids ne sert qu'une fois — ne pouvait pas tester du tout.

Avant de mesurer : trois erreurs dans du code jamais exécuté

Le code du lot 2 traitait déjà, sur le papier, les couches à poids partagés ; simplement, aucun modèle ne l'avait jamais fait tourner. Écrire d'abord les tests, puis les lancer contre ce code, a révélé trois erreurs, chacune mesurée plutôt que soupçonnée :

Erreurs trouvées et corrigées avant toute mesure

- La variante *reduce* de K-FAC sommais les entrées sur les positions là où la définition publiée en prend la *moyenne* : le résultat était exactement T^2 fois trop grand (T = nombre de positions). Mesuré sur deux cas-jouets : erreur relative de 24,000 pour $T = 5$ et de 80,000 pour $T = 9$, soit précisément $T^2 - 1$. Le biais allait dans le sens qui *avantage* l'autre variante — c'est-à-dire exactement la comparaison que ce lot devait trancher.
- La version « à valeurs propres corrigées » de cette même variante projetait une quantité qui n'est le gradient de rien, ce qui lui faisait perdre la propriété de conservation d'énergie qui la définit.
- TKFAC, sur ces couches, utilisait une troisième formule ne conservant l'énergie d'aucun objet défini, et différente de celle que l'optimiseur du dépôt applique réellement.

Les tests correspondants échouent sur l'ancien code et passent sur le nouveau. Aucun résultat antérieur n'est affecté : ces trois branches ne s'activent que lorsqu'un poids est partagé, ce qui n'arrive jamais sur le modèle A1.

Deux autres défauts ont été corrigés au passage. Sur le transformeur, 2 048 des 21 098 paramètres (les positions apprises) étaient absents de la matrice de référence, laquelle était malgré tout étiquetée « la vraie courbure » ; ils sont désormais inclus. Et chaque construction de référence se vérifie maintenant elle-même contre le calcul automatique de gradients, de deux façons complémentaires — la somme sur tous les exemples, et un exemple isolé — la seconde attrapant ce que la première ne peut pas voir. Sur les trois modèles réels, l'accord est de l'ordre de 5×10^{-16} , c'est-à-dire la précision de la machine.

Ce que les trois calculs ont produit

Trois calculs GPU, tous terminés sans incident (1 h 02 pour le transformeur, 4 h 21 et 4 h 31 pour les deux variantes du convolutif), chacun sur les **45 000** images d'entraînement, aux cinq instantanés et pour les deux sources. Point important pour la lecture : à cette taille d'échantillon, le plancher de bruit tombe à 0,005–0,039 selon le modèle et l'instantané, contre 0,343 sur le modèle A1. Les écarts rapportés ci-dessous lui sont dix à cent fois supérieurs — la règle de lecture « en dessous du bruit, on ne conclut pas » ne mord donc plus ici.

Les règles de décision avaient été **écrites et figées avant de voir la moindre donnée**, précisément pour qu'aucun seuil ne puisse être ajusté après coup.

HF2 tranchée, dans le même sens sur les trois modèles

(ii) Le choix *reduce* est le plus fidèle. En comptant chaque couple (couche partagée, instantané) comme un vote, avec son intervalle de confiance :

Modèle	cas	<i>reduce</i>	<i>expand</i>	amplitude des victoires de <i>reduce</i>
A2 (normalisation par groupes)	15	14	1	−0,71 à −0,15
A2 (BatchNorm)	15	15	0	−0,72 à −0,11
A3 (transformeur)	45	40	5	−0,94 à −0,08

Les rares victoires de *expand* sont petites (+0,02 à +0,18) et concentrées sur les instantanés les plus précoces, à une exception près : la couche qui découpe l'image en morceaux, à l'initialisation seulement (+0,78) — la seule couche dont les « positions » sont des morceaux d'image disjoints plutôt qu'une séquence que le réseau agrège ensuite.

(i) L'erreur de structure est bien plus grande sur les couches à poids partagés. L'erreur médiane de K-FAC y vaut 0,95 à 0,98, contre 0,005–0,011 à l'initialisation et 0,25–0,64 en fin d'entraînement sur la tête de classification, qui n'a pas de partage.

Mais le résultat le plus important n'était pas la question posée. La décomposition décrite en section 5.3 sépare l'erreur de K-FAC en deux étapes : ignorer les corrélations entre positions différentes, puis supposer l'indépendance entre entrées et gradients. Mesure : la première étape coûte à elle seule une fraction médiane de **0,971** — **identique sur les trois modèles** (étendue 0,78 à 0,995), la seconde 0,45 à 0,60 de ce qui reste. Autrement dit, sur une couche à poids partagés, l'erreur de K-FAC est **d'abord une erreur de partage de poids**, et non l'hypothèse d'indépendance dont la littérature discute habituellement.

Un fait complémentaire pointe dans la même direction : à l'initialisation, le bloc exact est *presque* de la forme que K-FAC suppose (la meilleure approximation de cette forme atteint 0,05 sur l'attention du transformeur et 0,22–0,37 sur les convolutions), alors que K-FAC lui-même reste à 0,95–0,97. Ce n'est donc pas la forme de Kronecker qui échoue, mais le choix particulier des deux facteurs que fait K-FAC.

Un avertissement à emporter avec HF2

Les deux façons de juger une approximation ne donnent pas le même gagnant. Sur l'ensemble des amortissements et des couches partagées, le choix *reduce* — plus fidèle en norme — produit un *moins bon* pas d'optimisation que *expand* dans 70/75, 62/75 et 162/225 des cas selon le modèle. Exemple chiffré (attention du transformeur, fin d'entraînement, amortissement léger) : erreur 0,73 contre 0,97 en faveur de *reduce*, mais efficacité du pas $\rho = 0,14$ contre 0,45 en faveur de *expand*. La règle de décision votait sur la fidélité et rapportait ρ séparément, précisément parce que les deux pouvaient diverger ; elles divergent. **Quiconque choisit entre ces deux variantes pour un optimiseur doit lire ρ , pas la fidélité.**

Les normalisations : la proposition 3.1 pointe dans la bonne direction, mais 20 à 38 fois trop bas

La comparaison entre la forme de Hadamard (la proposition 3.1 de l'article AdaFisher) et la simple diagonale exacte s'inverse entre A1 et les trois modèles de ce lot. L'explication est mesurée, et c'est une pure question d'échelle : le facteur multiplicatif optimal de la forme de Hadamard vaut 0,95 sur A1 (aucun partage de poids) mais 27,8, 20,4 et 37,9 sur les trois modèles à poids partagés. En direction — c'est-à-dire une fois ce facteur d'échelle neutralisé — le classement est identique sur les quatre modèles. La proposition 3.1 décrit donc correctement la *forme* de la courbure d'une couche de normalisation, mais en sous-estime la *taille* d'un facteur 20 à 38 dès qu'un poids est appliqué à plusieurs positions, pour une raison structurelle : elle

est bâtie sur des statistiques qui ignorent les corrélations entre positions, et approxime donc le bloc *déjà amputé* de ces corrélations.

Conséquence directe pour le dépôt : un estimateur corrigé pour les normalisations — piste que le plan de campagne identifie comme contribution possible — doit corriger l'échelle, et pas seulement réintroduire le terme croisé $\gamma\text{--}\beta$. Second point, désormais mesuré sur le bon objet : la formule effectivement livrée dans le code est la plus faible des cinq lectures testées, y compris en direction (dernière dans 13 cas sur 15 pour la variante BatchNorm, 15 sur 25 pour le transformeur).

Deux conclusions du lot précédent ne survivent pas au changement de modèle

Le critère de sortie du lot exigeait que toute conclusion « par type de couche » tienne sur au moins deux modèles. Deux n'y parviennent pas, et sont consignées comme telles :

Conclusion	A1	A2 / A3	Verdict
Le raccourci diagonal d'AdaFisher coûte quelque chose (HF3)	confirmée	confirmée	tient
Le terme croisé $\gamma\text{--}\beta$ n'est pas négligeable (HF4)	confirmée	confirmée	tient
Toutes les approximations se dégradent au fil de l'entraînement	95 % des cas	52 à 71 %	tendance, pas règle
Les couches sont fortement couplées entre elles (HF5)	minimum 0,63	0,28 (transformeur), 0,44 (convolutif)	ne tient pas

La dernière ligne est instructive dans sa direction : les couches sont *moins* liées entre elles là où les poids sont partagés que dans un simple perceptron multicouche. Ignorer les corrélations inter-couches — ce que font toutes les méthodes de ce dépôt — est donc *mieux* justifié sur une convolution ou un transformeur que sur le modèle jouet où on l'avait d'abord mesuré : exactement l'inverse de ce qu'une étude limitée au perceptron aurait laissé croire.

Quant aux amplitudes, elles valent la peine d'être citées : le coût du raccourci diagonal atteint 0,86–0,90 sur les convolutions et 0,83–0,97 sur les couches du transformeur en fin d'entraînement (contre 0,25–0,41 sur les têtes de classification), soit un ordre de grandeur au-delà du seuil de 10 % fixé par l'hypothèse ; et le terme croisé des normalisations porte 0,65–0,70 de l'énergie du bloc sur le convolutif, 0,39–0,62 sur le transformeur.

14.5 Un problème numérique corrigé en cours de route : les valeurs propres exactement nulles

Une conséquence directe et concrète du fait que F possède des directions de rang nul (comme le noyau de $d_{\text{in}} + 1$ dimensions de la tête de classification, ou les pixels de bordure identiquement nuls de MNIST évoqués plus haut) s'est manifestée pendant les campagnes d'entraînement elles-mêmes, avant même le début de cette campagne de mesure : la décomposition en valeurs propres nécessaire aux modes **ekfac**/**tekf** peut échouer brutalement sur le GPU quand la matrice à décomposer est *exactement* déficiente en rang (et non pas seulement « mal conditionnée » au sens usuel), un cas que deux campagnes d'entraînement réelles ont effectivement rencontré. La correction (un petit terme de régularisation ajouté avant la décomposition, mathématiquement inerte sur le résultat final puisqu'il ne fait que décaler uniformément les valeurs propres sans toucher aux vecteurs propres) est maintenant en place, et ce même phénomène de rang exactement nul est aujourd'hui *quantifié* plutôt que seulement contourné, grâce au calcul dense complet décrit ci-dessus.

15 Feuille de route : les lots d'implémentation restants

Le travail de cette campagne est découpé en lots successifs, chacun construit sur les précédents et validé par ses propres tests avant de passer au suivant — le même principe de « une itération à la fois » qui a déjà structuré la construction des cinq modes de l'optimiseur.

Lot	Contenu	Où ça en est
0	Les fondations : politique de précision numérique, désactivation du mode de calcul approché <i>TF32</i> , ensembles d'exemples sondes reproductibles, registre qui classe chaque paramètre de chaque modèle par type de couche, et le pont qui sait lire les instantanés déjà sauvegardés par les bancs d'essai.	Terminé : 27 tests dédiés, aucun changement au code existant.
1	Le premier modèle traité de bout en bout, en régime dense : capture des gradients par exemple, les trois sources (type-2, Monte-Carlo, empirique), et le calcul exact de F , $\hat{E}$ et des blocs par couche, en double précision.	Terminé, sauf le calcul complet à grande échelle , qui est lui-même désormais terminé (section 14).
2	Le « zoo » complet des approximations (bloc-diagonal, K-FAC, EK-FAC, TKFAC, diagonale) et les métriques M1, M3, M5, M7, M8, appliqués au premier modèle, sur ses cinq instantanés ; premier tracé du plancher de bruit.	Terminé (section 14.3) ; deux lectures de ses résultats ont dû être corrigées par le lot 3.
3	Extension à un modèle avec convolutions et normalisations (dans ses deux variantes), et à un modèle avec attention ; la décomposition partage de poids / indépendance ; intervalles de confiance par couche.	Terminé (section 14.4) : HF2 tranchée sur trois modèles, trois erreurs de code antérieures corrigées au passage, et deux conclusions du lot 2 invalidées.
4	Le régime B (couche par couche, sans jamais former la grande matrice), validé d'abord contre le régime dense, puis appliqué aux modèles plus grands — dont l'auto-encodeur MNIST et son blocage (section 14.2).	À venir ; c'est le lot qui doit trancher la cause du blocage.
5	Le protocole opérationnel P2 : lecture directe de l'état réel de l'optimiseur, avec le réchauffage déjà quantifié (section 8).	À venir.
6	Extensions optionnelles : un modèle à largeur réaliste, un modèle en régime sans matrice, et d'éventuels nouveaux modèles pour couvrir les types de couches encore absents (<i>embeddings</i> de texte, LoRA).	Optionnel, décidé après le lot 5.

L'ordre n'est pas arbitraire : le régime dense (lot 1–3) sert de **juge de référence** pour valider le régime factorisé du lot 4 avant de lui faire confiance sur des modèles où aucun calcul dense n'est possible ; et le protocole opérationnel (lot 5) a besoin du zoo d'approximations déjà construit

et validé par le lot 2. Créer un tout nouveau modèle de banc d’essai (lot 6) est délibérément placé en dernier, car cela représenterait un changement dans les bancs d’essai eux-mêmes, que cette campagne s’est fixé comme règle de ne faire qu’en tout dernier recours.

16 Risques identifiés et façons de les traiter

- **Le rang déficient du régime B.** Comme expliqué section 6, dans le régime factorisé, toute métrique qui passe par un inverse mélange une vraie erreur d’approximation avec de la courbure « inventée » là où la référence n’a simplement aucune information. Parade : toujours présenter ces métriques comme des courbes en fonction de l’amortissement λ , jamais comme un point unique, et confronter au régime dense chaque fois que c’est possible.
- **La précision numérique.** Les spectres de courbure s’étalent sur plus de huit ordres de grandeur ; toutes les références et les Grams intermédiaires sont donc calculés en double précision (*fp64*), et le mode de calcul approché *TF32* des GPU récents (qui introduirait une erreur relative de l’ordre de 10^{-3} , suffisante pour fausser tous les tests d’exactitude) est explicitement désactivé et vérifié par un test dédié.
- **Un piège matériel concret.** Les tâches d’entraînement du dépôt tournent sur une petite tranche de GPU (10 Go de mémoire) ; une tâche d’analyse qui hériterait de cette même contrainte de mémoire ne pourrait tout simplement pas former F pour les modèles en régime dense. Chaque tâche d’analyse générée déclare donc explicitement son propre besoin en mémoire, plus généreux.
- **Des instantanés mal étiquetés.** Une correction apportée en cours de campagne au calcul des fractions d’entraînement (0 %, 1 %, ...) a rendu incorrects tous les instantanés produits *avant* cette correction, à l’exception de ceux du mode *diag*. Le pont de lecture des instantanés refuse activement de charger un instantané dont les métadonnées indiquent qu’il provient d’avant cette correction, plutôt que de risquer une comparaison silencieusement faussée.
- **Des graines encore en cours d’exécution.** Comme signalé section 11, l’analyse doit systématiquement rapporter le nombre de graines réellement disponibles au moment de la mesure, jamais un nombre visé par avance.
- **Le volume de stockage.** Les matrices intermédiaires du régime B (les « Grams » par couche) ne sont jamais écrites sur disque, seules les métriques finales le sont — sans quoi le volume de données produites deviendrait rapidement ingérable.

17 Ce que la campagne produira, et à quel coût

17.1 Le format des résultats

Chaque combinaison (modèle, bras, graine, fraction d’entraînement) produit un tableau de résultats au format *CSV*, avec une ligne par mesure (une couche, une structure d’approximation, une source, une référence, une valeur d’amortissement, une métrique, sa valeur) et un petit fichier d’accompagnement qui trace, pour chaque tableau, la version exacte du code utilisée, les exemples sondes précis employés, et les métadonnées de l’instantané d’origine — de sorte que tout résultat reste entièrement reproductible et vérifiable après coup.

17.2 Les figures prévues

- une carte de l’efficacité ρ , croisant chaque structure d’approximation et chaque type de couche, une par référence (F ou $\hat{E}$) ;

- un diagramme en barres décomposant l’erreur totale en ses trois composantes (source, structure, interaction) ;
- des courbes suivant l’évolution de la fidélité au fil de l’entraînement, séparément pour les sondes d’entraînement et de validation ;
- une carte du biais d’indépendance (σ_2/σ_1) par couche ;
- des matrices de couplage entre couches différentes ;
- un nuage de points comparant, couche par couche, le classement obtenu en conditions idéalisées (P1) contre le classement obtenu en conditions réelles (P2) ;
- une courbe du plancher de bruit en fonction du nombre d’exemples sondes utilisés ;
- un panneau dédié spécifiquement aux couches de normalisation.

17.3 Le budget de calcul

Le calcul dense sur le premier modèle (A1) a été mesuré à un peu plus d’une heure sur une tranche de GPU, avec un pic de mémoire de 6,06 Go — très proche de la prévision théorique de 6,0 Go faite avant de lancer la tâche, ce qui donne confiance dans les estimations de coût pour les modèles restants. La campagne principale complète (les modèles en régime dense et factorisé, cinq instantanés, les graines disponibles, les deux protocoles P1 et P2) est estimée à l’ordre de 100 à 200 heures de calcul GPU sur la tranche de matériel utilisée pour l’analyse.

18 Glossaire rapide

Terme	Signification en clair
Matrice de Fisher F	la courbure exacte du problème d’apprentissage, moyennée sur les étiquettes que le modèle imagine lui-même possibles.
Fisher empirique $\hat{E}$	la même idée, mais moyennée seulement sur les étiquettes réellement observées dans les données — bien moins chère à calculer, structurellement biaisée.
GGN (Gauss–Newton généralisée)	une autre courbure classique, mathématiquement identique à F dans tous les cas pratiques utilisés dans ce dépôt.
K-FAC	une façon d’approcher la courbure d’une couche en la factorisant en un produit de deux matrices bien plus petites (une pour les entrées de la couche, une pour ses gradients de sortie).
EKFAC	K-FAC, en réajustant après coup l’importance de chaque direction de façon optimale.
TKFAC / TEKFAC	une autre factorisation, construite pour préserver exactement l’énergie totale (la trace) de la courbure exacte.
Amortissement (λ)	un petit nombre ajouté à la diagonale d’une matrice pour pouvoir l’inverser sans risque, même si certaines de ses valeurs propres sont nulles ou très petites.
EMA (moyenne mobile exponentielle)	une façon de lisser une mesure dans le temps, en mélangeant la nouvelle observation avec la mémoire des observations précédentes.
Régime A / B / C	trois façons de calculer la courbure exacte selon la taille du modèle : tout former explicitement (A), ne jamais former la grande matrice mais garder des Grams par couche (B), ou ne calculer que des produits matrice-vecteur sans jamais rien former (C).
Protocole P1	mesure de la meilleure fidélité <i>possible</i> d’une approximation, dans des conditions idéalisées.

Terme	Signification en clair
Protocole P2	mesure de la fidélité <i>réellement obtenue</i> pendant un entraînement réel, avec tous les compromis pratiques empilés.
Réchauffage (re-warm)	le fait de relancer un optimiseur neuf pendant quelques centaines de pas, à partir d'un instantané qui ne contient que les poids, pour reconstituer une mémoire interne représentative avant de la mesurer.
Plancher de bruit	la variabilité statistique minimale, inévitable, d'une mesure faite sur un nombre fini d'exemples — en dessous de laquelle une différence n'est pas significative.
Checkpoint / instantané	une sauvegarde des poids d'un réseau à un instant donné de son entraînement.
Arm / bras	une des sept stratégies d'entraînement comparées (les cinq modes de courbure, plus Adam, plus AdamW).